



ТЕХНОЛОГИЧНО УЧИЛИЩЕ "ЕЛЕКТРОННИ СИСТЕМИ"
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

ДИПЛОМНА РАБОТА

по професия код 481020 „Системен програмист“
специалност код 4810201 „Системно програмиране“

Тема:

Уеб приложение за многостепенно обхождане на уеб ресурси и интелигентно извличане и обработка на информация чрез изкуствен интелект

Дипломант:

Дамян Иванов Мяшков

Дипломен ръководител:

Теодора Йовчева

СОФИЯ
2026



ТЕХНОЛОГИЧНО УЧИЛИЩЕ "ЕЛЕКТРОННИ СИСТЕМИ"
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

Дата на заданието: 17.10.2025 г.

Дата на предаване: 17.01.2026 г.

Утвърждавам:.....

/проф. д-р инж. П. Якимов/

ЗАДАНИЕ за дипломна работа

ДЪРЖАВЕН ИЗПИТ ЗА ПРИДОБИВАНЕ НА ТРЕТА СТЕПЕН НА ПРОФЕСИОНАЛНА КВАЛИФИКАЦИЯ
по професия код 481020 „Системен програмист“
специалност код 4810201 „Системно програмиране“

на ученика Дамян Иванов Мяшков от 12В клас

1. Тема: Уеб приложение за многостепенно обхождане на уеб ресурси и интелигентно извличане и обработка на информация чрез изкуствен интелект
2. Изисквания:
 - 2.1. Автоматизирано обхождане на уеб ресурси
 - 2.2. Интерфейс за работа с извлечената информация чрез изкуствен интелект
 - 2.3. Визуализация на процеса на обхождане на уеб страниците
3. Съдържание
 - 3.1 Теоретична част
 - 3.2 Практическа част
 - 3.3 Приложение

Дипломант :.....

/ Дамян Мяшков /

Ръководител:.....

/ Теодора Йовчева /

ВРИД Директор:.....

/ ст. пр. д-р Веселка Христова /



ТЕХНОЛОГИЧНО УЧИЛИЩЕ "ЕЛЕКТРОННИ СИСТЕМИ"
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

СТАНОВИЩЕ към ДИПЛОМНА РАБОТА

Тема на дипломната работа: Система за дълбоко многостепенно обхождане на уеб сайтове и интелигентно извличане и преработване на информация чрез изкуствен интелект

Ученик: Дамян Иванов Мяшков

Клас: 12 В

Професия: код 481020 „Системен програмист“

Специалност: код 4810201 „Системно програмиране“

Дипломен ръководител: Теодора Кирилова Йовчева

Дипломантът е изпълнил всички поставени задачи свързани с реализацията на дипломния проект и се е справил отлично с процеса на разработка.

Дамян сам избра темата на дипломната си работа. Той подходи отговорно към анализа на различните съвременни технологии, които може да използва. Всички нужни функционалности заложи в Заданието на дипломния проект са имплементирани и е описал всяка стъпка от работния процес - от проучването до самата разработка.

Дипломантът съвестно и отговорно отдели време да разучи новите за него технологии с онлайн материали и видеа. Винаги е имал точни въпроси, които да зададе по време на нашите срещи и винаги имаше прогрес, който можеше да покаже от предишната ни среща или разговор до тогавашния момент.

Предлагам за рецензент: Валери Николаев Добрев, vdobrev@elsys-bg.org, 0888370965.

Всичко изложено дотук дава основание ученикът Дамян Мяшков да бъде допуснат до защита пред комисията за провеждане на държавен изпит за придобиване на професионална квалификация по теория и практика на специалността и комисията да оцени дипломната работа отлично.

23.02.2026 г.

Дипломен ръководител:.....

/Теодора Йовчева/



Увод

В съвременната информационна среда достъпът до знания е по-лесен от всякога. Само с няколко търсения можем да достигнем до огромно количество уеб съдържание – корпоративни сайтове, техническа документация, научни публикации, медийни материали и анализи. Парадоксалното обаче е, че именно изобилието от информация затруднява процеса на систематично и задълбочено проучване. Често изследването се превръща в отваряне на десетки табове, частично прочитане на текстове и липса на ясна връзка между отделните източници. Проверяването на конкретно твърдение изисква допълнително време, а проследяването му до първоначалния текст откъс нерядко остава неизпълнено.

Появата на големите езикови модели значително улесни ориентацията в информацията. Те позволяват задаване на въпроси и предоставят синтезирани, структурирани отговори, които обединяват данни от различни източници. Това ги превръща в удобен инструмент за първоначална ориентация и обобщаване на теми. Въпреки това, при по-задълбочено уеб проучване възникват ограничения, свързани с обхвата на анализираната информация, степента на доказателствена проследимост и контрола върху използваните източници.

Например при анализ на корпоративен сайт с десетки продуктови и проектни подстраници, потребителят може да подаде линка към началната страница на езиков модел и да получи структурирано резюме. В повечето случаи обаче моделът не обхожда систематично цялата вътрешна структура на сайта, а анализира ограничен брой страници, достъпни чрез видимото съдържание или отделни резултати от търсене. При въпроси, изискващи цялостна представа, като „обобщение на дейността“, това почти неизбежно води до частичен обхват на информацията, без потребителят да разполага с механизъм да установи кои страници не са били разгледани.

Когато част от съдържанието липсва, моделът формулира логически последователен синтез въз основа на наличните данни. Така могат да бъдат изградени обобщения и връзки, които звучат убедително, но не са изрично подкрепени от всички релевантни източници. Допълнително информацията често се перифразира и обобщава, а като референция се посочва обща страница, което



затруднява проследяването на точния текстов откъс. В резултат може да се създаде привидно завършен и обоснован анализ, който реално се основава на непълен набор от данни и трудно проверими твърдения.

Подобна ситуация има особено значение в контексти, в които се изисква висока степен на точност и проверимост. Инвеститор, който оценява дейността на компания; журналист, който сравнява позиции по даден въпрос; юрист, който анализира условия в договор; или изследовател, който обобщава техническа информация, разчитат на пълнота на обхвата и възможност за ясна проследимост на всяко твърдение. Липсата на систематично обхождане и трудно проверимата аргументация могат да доведат до погрешни изводи, без това да бъде очевидно в самия отговор.

Дипломната работа разглежда този проблем и предлага решение чрез разработването на система, наречена Scholia. Scholia представлява интелигентна среда за изследване на уеб съдържание, която автоматично обхожда подадени уеб адреси в зададени граници, събира свързаните страници и индексира тяхното съдържание. Информацията се обработва и структурира така, че да може да бъде търсена и анализирана в контролиран контекст.

При задаване на въпрос системата извлича релевантни текстови откъси от вече събраната база от страници и формулира отговор единствено въз основа на този корпус. Всяко твърдение се свързва с конкретен текстов откъс и оригинален източник, което осигурява прозрачност и възможност за незабавна проверка. По този начин се цели преодоляване на ограниченията, свързани с частичния обхват, вероятностното обобщаване и трудната проследимост на информацията.

Целта на дипломната работа е да се проектира и реализира функционално уеб приложение, което автоматизира процеса по събиране, структуриране и интелигентно търсене в уеб съдържание, като осигурява контролируем обхват на анализа и ясна проследимост на използваните източници. За постигането на тази цел се поставят задачи, свързани с проучване на съществуващи решения, дефиниране на изисквания към системата, проектиране на архитектурата, реализиране на алгоритмите за обхождане и индексиране, както и разработване на потребителски интерфейс за взаимодействие със събраната информация.



Първа глава: Съществуващи решения

1.1. Google / Bing: Стандартното проучване в уеб среда (процес и проблеми)

В съвременната уеб среда процесът на проучване обикновено започва с използване на търсачка или директно посещение на конкретен сайт. Потребителят отваря начална страница, преглежда наличната информация и последователно преминава към свързани подстраници чрез хипервръзки. Този процес е изцяло ръчен и зависим от избора на потребителя кои връзки да бъдат отворени и кои части от съдържанието да бъдат счестени за релевантни.

При сайтове с по-сложна структура като корпоративни портали с десетки продуктови страници, проектни секции и архиви, този подход води до фрагментиране на информацията. Съдържанието остава разпределено между множество отделни страници, а потребителят трябва сам да обединява данните в цялостна картина.

На първо място, липсва гаранция за пълен обхват на проучването. Потребителят трудно може да бъде сигурен, че е достигнал до всички релевантни подстраници. Част от информацията може да бъде скрита в по-дълбоки нива на навигацията или в слабо видими връзки. Това създава риск от формиране на изводи върху непълен набор от данни.

На второ място, проверката на конкретни твърдения е трудоемка и времеемка. За да потвърди дадена информация, потребителят трябва ръчно да открие съответния текстов откъс, да го сравни с други източници и да проследи неговия контекст. При по-голям обем съдържание този процес се усложнява значително.

На трето място, липсва автоматизирана структурна връзка между отделните страници. Потребителят сам изгражда ментален модел на съдържанието, определя кои елементи са свързани и как отделните твърдения се отнасят едно към друго. Когнитивната натовареност нараства пропорционално на обема и сложността на информацията.

В резултат стандартното уеб проучване остава процес, който изисква значителни усилия, внимание и време. Тези ограничения пораждат необходимост



от инструменти, които да подпомагат автоматизирането на анализа и структурираното обединяване на информацията. В следващата точка ще бъдат разгледани съвременни системи, които се използват с подобна цел, както и техните възможности и ограничения.

1.2. Подобни системи и продукти

1.2.1. ChatGPT, Claude & Gemini – Стандартни големи езикови модели като инструмент за проучване: възможности и ограничения

Големите езикови модели като ChatGPT, Claude, Gemini и сходни системи се утвърдиха като универсални инструменти за достъп до информация. Те предоставят диалогов интерфейс, който позволява на потребителя да формулира въпроси и да получава синтезирани, логически структурирани отговори. В контекста на проучването тези системи предлагат значително улеснение спрямо класическото търсене, тъй като обединяват информацията в един последователен текстов анализ, вместо да изискват ръчно преглеждане на множество страници.

Основното им предимство е способността за бърза агрегация и обобщаване на съдържание. Потребителят може да зададе комплексен въпрос, изискващ съпоставка между различни аспекти на дадена тема и да получи структуриран отговор в рамките на секунди. В някои случаи моделите разполагат и с възможност за достъп до уеб търсене, което позволява извличане на актуална информация и посочване на външни източници.

Въпреки тези предимства, при използването им като инструмент за систематично уеб проучване възникват съществени ограничения.

1.2.1.1. Пример, в който излизат наяве недостатъците

За да се илюстрира недостигът на тези системи в изследователски контекст, нека разгледаме конкретна ситуация.

Да си представим, че се намирате в позицията на инвеститор, който обмисля вложение в компания за международна търговия с картофени семена. Преди да вземете решение, искате да получите пълна картина на дейността ѝ: какви са продуктите, на кои пазари оперира, има ли регулаторни ограничения, как са структурирани проектите ѝ и какви са реалните ѝ конкурентни предимства.

Сайтът на компанията съдържа начална страница, двадесет отделни подстраници за различни сортове, страница с десет ключови проекта, всеки със



собствено подробно описание, раздел за международни пазари и регулации, както и информация за лицензиране и партньорства. Част от тази информация се намира в по-дълбоки нива на навигацията и не е директно достъпна от началната страница.

Вие подавате линка към началната страница на стандартен езиков модел и задавате въпрос:

„Обясни дейността на компанията, основните ѝ пазари и начина, по който всеки от продуктите ѝ са разпределени по региони.“

В този момент са възможни два различни сценария.

В първия сценарий моделът може да заяви, че няма достъп до цялата вътрешна структура на сайта и че разполага само с ограничена информация. Това е по-консервативният отговор. В този случай обаче потребителят трябва ръчно да подава допълнителни линкове към всяка продуктова страница, всяка проектна секция и всяка регулаторна информация. Проучването се превръща в бавен и трудоемък процес, при който инвеститорът трябва да навигира модела през сайта стъпка по стъпка. Системата не извършва самостоятелно систематично обхождане, а реагира единствено на конкретни указания.

Вторият сценарий, който на практика се среща по-често, е моделът да формулира завършен и уверен синтез въз основа на частично достъпната информация. Той анализира началната страница и евентуално ограничен брой допълнителни линкове от уеб търсене, след което представя структуриран отговор от типа:

„Компанията предлага диверсифицирано портфолио от двадесет сорта картофени семена и оперира на множество международни пазари чрез глобална мрежа от партньори.“

Отговорът е логически последователен, стилово професионален и създава усещане за цялостен анализ на бизнес модела. За потребителя не е очевидно кои страници са били разгледани и кои не.

Проблемът се проявява при задаване на по-конкретен въпрос:

„Предлагат ли се всички сортове на всички пазари?“

На база вече изграденото обобщение моделът може да заключи, че продуктите се разпространяват глобално. В действителност обаче в



подстраницата на определен сорт може да е посочено, че той се предлага само в рамките на ЕС поради фитосанитарни ограничения. В друг раздел може да е уточнено, че част от продуктите са налични само чрез лицензирани партньори и не се изнасят директно.

В този момент моделът е изградил логическа връзка между частични данни и я е представил като завършено заключение. Той не сигнализира, че анализът не е обхванал всички двадесет продуктови подстраници. За инвеститора това представлява реален риск. Въз основа на убедителния синтез той може да направи погрешна оценка за степента на пазарно покритие на компанията.

Допълнително усложнение възниква при проверката, ако като източник е посочена началната страница. В нея може да присъстват общи формулировки като „международно присъствие“ и „широко портфолио“, но не и конкретното заключение за глобална наличност на всички продукти. Поради перифразиране и обобщаване конкретното твърдение не може лесно да бъде проследено до точен текст откъс. Така логическото допълване остава скрито зад формално коректно цитиране.

В този сценарий моделът не просто пропуска информация, той създава убедителна илюзия за изчерпателен анализ. Потребителят не получава сигнал за непълнота и може да приеме синтеза като напълно основан на доказателства, докато реално част от релевантните страници изобщо не са били разгледани.

1.2.1.2. Структурни ограничения

След разгледания пример могат да бъдат формулирани няколко ясно разграничими структурни ограничения, които произтичат от начина, по който функционират големите езикови модели като инструмент за уеб проучване. Тези ограничения не са единични грешки или неточности, а системни характеристики, които се проявяват при сценарии, изискващи изчерпателност, контрол и доказателствена проследимост.

Първото ограничение е липсата на гарантиран и измерим обхват на анализа. Моделът не извършва систематично обхождане на вътрешната структура на даден сайт. Дори когато разполага с уеб достъп и показва използваните източници, това представлява списък единствено на страниците, до които е достигнал, а не доказателство, че всички релевантни подстраници са били анализирани.



Потребителят няма механизъм да зададе граници на обхвата, да изисква обход на целия домейн или да провери кои части от сайта са останали неизследвани. При въпроси от типа „Обобщи целия сайт“ или „Анализирай всички продукти“, системата не гарантира, че реално са разгледани всички страници, които съдържат съществена информация. Това означава, че липсва измерима пълнота на анализа. Потребителят не знае дали получава изчерпателен преглед или само фрагмент от наличното съдържание.

Второто ограничение е вероятностното запълване на информационни пропуски. Когато моделът работи с частичен набор от данни, той изгражда най-вероятния логически синтез между наличните фрагменти. Това произтича от самата му архитектура, която е насочена към предсказване на най-вероятното продължение на текста въз основа на статистически зависимости. При непълен контекст обаче това води до изграждане на връзки, които изглеждат логически убедителни, но не са изрично потвърдени от всички релевантни източници. Моделът не отбелязва ясно, че заключението е изведено при ограничен обхват на данните. В резултат потребителят възприема синтеза като завършен и аналитично обоснован, въпреки че част от него може да представлява вероятностна интерпретация, а не проверен факт. Това създава риск от формиране на погрешни изводи, които изглеждат стабилни, но се основават на непълна база.

Третото ограничение е неясната граница между директно извлечена информация и интерпретация. В генерирания отговор моделът не маркира кои части представляват директно подкрепени твърдения и кои са резултат от обобщение или логическо свързване. От гледна точка на потребителя целият текст има еднаква степен на достоверност. Липсва ясно разграничение между „това е написано в източника“ и „това е изведено като логически синтез“. При изследователски сценарии това е съществен проблем, тъй като доказателствената тежест на директен текстов откъс и на интерпретативно заключение не е равностойна. Без експлицитно разграничение потребителят трудно може да прецени кои части от анализа изискват допълнителна проверка.

Четвъртото ограничение е свързано с доказателствената проследимост и проверимостта на заключенията. Поради перифразиране и обобщаване конкретните формулировки в отговора често не съществуват в оригиналния текст в същия вид. Посочването на обща страница като източник не гарантира, че всяко



отделно твърдение може лесно да бъде проследено до конкретен текстов откъс. Потребителят трябва ръчно да търси в посочения източник дали формулираното заключение действително се подкрепя от съдържанието. Това увеличава когнитивната натовареност и частично обезсмисля предимството на автоматизирания синтез. При сложни сайтове с голям обем съдържание подобна проверка може да бъде практически невъзможна без допълнителни инструменти.

В обобщение, стандартните големи езикови модели са изключително ефективни при бърза агрегация и структуриране на информация. Те обаче не осигуряват гарантиран обхват на анализираното съдържание, не разграничават ясно факт от вероятностна интерпретация и не предоставят пълна доказателствена проследимост на всяко заключение. Тези ограничения не са случайни технически несъвършенства, а произтичат от самия принцип на функциониране на модела. Именно поради това при сценарии, в които се изисква систематично, контролируемо и проверимо уеб проучване, възниква необходимост от специализирана система, която да осигури автоматизирано обхождане (crawling), структурирано индексирание и директна връзка между всяко твърдение и конкретен доказателствен текстов откъс.

1.2.2. Специализирани LLM системи с предварително подбран или качен набор от документи

В предходната точка бяха разгледани общите големи езикови модели, които работят като универсални диалогови агенти с ограничен и неконтролируем обхват на уеб достъп. Като реакция на тези ограничения се появиха системи, които комбинират езиков модел с ясно дефиниран набор от текстове чрез механизма Retrieval-Augmented Generation.

Тези решения се различават от стандартните чат модели по това, че не разчитат на произволно уеб търсене, а работят върху предварително определен набор от документи. Обикновено текстовете се разделят на смислови сегменти, създават се векторни представления, а при задаване на въпрос се извличат най-релевантните откъси, върху които моделът формулира отговор. Това осигурява по-голям контрол върху използваните източници и намалява вероятността от въвеждане на външна, непроверена информация.

1.2.2.1. Домейн-специфични GPT системи



Първата подкатегория включва специализирани GPT системи, разработени за конкретна област. Такива могат да бъдат маркетингови анализатори, правни асистенти, медицински помощници, агрономически консултанти или вътрешни корпоративни чат системи. Те работят върху предварително подбран набор от фирмени документи, продуктови описания, регулаторни текстове или вътрешна база знания.

Тези системи почти винаги използват RAG архитектура. Предварително събраните текстове се обработват, индексират се чрез embeddings и при въпрос се извличат най-подходящите откъси. Това означава, че те решават част от проблемите, разгледани в 1.2.1. По-конкретно, намаляват вероятността от халюцинации извън конкретния набор от източници и ограничават анализа до ясно дефинирани текстове.

Основното им предимство е контролът върху използваната информация. Потребителят знае, че отговорите се базират на определен, предварително подбран набор от документи. Ограничението обаче е също толкова ясно: този набор е статичен. Системата не извършва динамично обхождане на произволен уебсайт и не изгражда нова структура от свързани страници според конкретната изследователска задача. Тя работи в рамките на вече подготвена среда.

1.2.2.2. Системи за чат върху качени документи

Втората подкатегория включва инструменти като AskYourPDF, ChatPDF, Numata и сходни решения, които позволяват на потребителя да качи един или повече документи и да задава въпроси върху тях. Тези системи също използват RAG механизъм. Качените файлове се разделят на текстови сегменти, създават се векторни представяния и при въпрос се извличат релевантните откъси, които служат като основа за генериране на отговор.

Този тип решения адресира важен проблем: осигурява напълно ясен обхват на анализираната информация. Потребителят знае точно кои документи са включени и няма неяснота относно източниците. Често се предоставят и директни цитати от конкретни страници, което подобрява проследимостта.

Ограничението тук е различно от това при домейн-специфичните системи. Потребителят сам трябва да избере и качи документите. Ако анализира сайт с десетки подстраници, той първо трябва ръчно да ги изтегли, конвертира или



обедини. Системата не извършва автоматично обхождане на уеб структурата, не проследява вътрешни връзки и не изгражда картина на взаимовръзките между страниците. Така първата фаза на събиране на информация остава изцяло ръчна.

1.2.2.3. Обобщение

Специализираните GPT системи и инструментите за чат върху качени документи представляват важна стъпка към по-контролируемо използване на езикови модели. Те ограничават анализа до ясно дефиниран набор от текстове и често осигуряват по-добра проследимост на източниците.

Въпреки това те не решават проблема със систематичното и автоматизирано обхождане на произволна уеб структура. Те работят върху предварително избран или ръчно предоставен набор от документи, а не върху динамично изследване на свързани страници според конкретната изследователска задача. Именно тук остава разликата между статичните RAG системи и необходимостта от инструмент, който съчетава автоматично събиране на уеб съдържание, структуриране и доказателствено обвързване на всяко заключение с конкретен текстов откъс.

1.2.3. Perplexity и сходни търсещо-ориентирани AI системи с цитиране на източници

Perplexity представлява система, която комбинира езиков модел с интегрирано уеб търсене и автоматично цитиране на използваните източници. За разлика от стандартните чат модели, при които уеб достъпът е допълнителна функционалност, тук процесът на извличане на външни страници е централен елемент от архитектурата. При всяко запитване системата извършва търсене в интернет, подбира релевантни резултати и генерира синтезиран отговор, като свързва отделни части от текста с конкретни линкове.

Този подход осигурява по-висока степен на прозрачност спрямо класическите LLM интерфейси. Потребителят получава видим списък от използвани страници и може директно да отвори източниците. Това намалява риска от напълно необосновани твърдения и създава усещане за доказателствена подкрепа на анализа. В сравнение със стандартните чат модели без цитиране, Perplexity предлага по-ясна връзка между генерирания текст и външните източници.



Въпреки това начинът на работа остава въпросно-ориентиран. При всяко ново запитване системата извършва ново търсене и изгражда отговор въз основа на ограничен набор от резултати, които алгоритъмът счита за най-релевантни спрямо формулирания въпрос. Не се изгражда предварително цялостна картина на конкретен сайт, домейн или ясно дефиниран информационен масив. Вместо това анализът се формира динамично около конкретната формулировка на запитването.

Това означава, че обхватът на използваната информация зависи силно от начина, по който е зададен въпросът. Различни формулировки могат да доведат до различни източници и съответно до различни акценти в отговора. Липсва механизъм, чрез който да се гарантира, че всички релевантни страници в рамките на определен сайт или определена тематична област са били разгледани. Системата оптимизира за релевантност спрямо въпроса, но не и за изчерпателност спрямо конкретен информационен обект.

Този модел е особено ефективен при широки теми и бърза ориентация в актуални събития, където целта е сравнение на множество външни източници и извличане на обобщена картина. При сценарии, изискващи систематичен анализ на ясно дефиниран набор от страници, например цялостно изследване на структурата на конкретен сайт, проследяване на всички продукти в портфолио или анализ на регулаторна документация в рамките на даден домейн, подходът на многократно търсене може да доведе до пропускане на релевантна информация, ако тя не бъде извлечена от алгоритъма за търсене при конкретната формулировка.

Въпреки наличието на цитати, генерираният текст остава синтез и интерпретация. Конкретните формулировки рядко съвпадат дословно с оригиналните източници, което означава, че проверката изисква допълнително усилие от страна на потребителя. Цитирането повишава прозрачността, но не осигурява автоматично гаранция за пълен обхват или ясно разграничение между директно извлечена информация и обобщаваща интерпретация.

В този контекст Perplexity може да бъде разглеждана като най-близкия до изследователска логика масово достъпен инструмент, тъй като комбинира търсене и цитиране в единна среда. Неговата архитектура обаче остава реактивна спрямо отделни въпроси, а не изградена около предварително систематично



обхождане и структуриране на конкретен информационен масив. Именно тук се проявява практическата разлика спрямо системи, които първо създават контролирана база от страници в рамките на зададен обхват, а след това позволяват многократно задаване на въпроси върху този вече фиксиран набор от данни.

При задачи, при които е необходим гарантиран обхват на конкретен домейн, ясна граница на анализирания съдържание и пряка връзка между всяко твърдение и точен текстов откъс от предварително събраните страници, подходът на предварително обхождане и индексване се оказва по-подходящ. Той не заменя динамичното уеб търсене, а адресира различен тип изследователска потребност – тази, при която приоритет са пълнотата на разглеждания обект и проследимостта на всяко заключение в рамките на ясно дефиниран информационен обхват.

1.2.4. Класически web scraping и crawler инструменти: възможности за интеграция с езикови модели

Преди появата на масово достъпните езикови модели (LLMs), автоматизираното събиране на уеб съдържание се реализира основно чрез класически crawler и web scraping инструменти. Тяхната функция е систематично обхождане на уеб страници, извличане на структурирана или неструктурирана информация и съхраняването ѝ в база данни за последващ анализ. Типични примери са инструменти и библиотеки като Scrapy, BeautifulSoup, Selenium, както и корпоративни crawler платформи, използвани за мониторинг на цени, конкурентен анализ или агрегиране на съдържание.

Тези технологии имат едно фундаментално предимство спрямо въпросно-ориентираните AI системи. Те позволяват пълен и контролиран обхват на даден сайт или домейн. При правилна конфигурация crawler-ът може да обхване всички вътрешни страници в рамките на зададени граници, да следва навигационната структура и да гарантира, че съдържанието е систематично събрано. За разлика от моделите, които избират ограничен брой страници на база релевантност към конкретен въпрос, crawler инструментът може да бъде настроен да покрие целия наличен информационен масив.

В практиката обаче тези инструменти често се използват в ограничени и строго функционални сценарии. Най-разпространените приложения са



автоматично извличане на цени от онлайн магазини, събиране на новинарски публикации или агрегиране на данни за маркетингови цели. Събраното съдържание обикновено се използва за статистически анализ, сравнителни таблици или мониторинг, но не и за интелигентна семантична обработка. Липсва интегриран слой, който да позволява естественоезиково взаимодействие със събраната информация.

Точно тук се очертава потенциалът на комбинирането на crawler технологии с езикови модели. Ако събраното съдържание бъде индексирано и структурирано в информационна база, върху него може да бъде изграден слой за семантично търсене и генериране на отговори. В такъв сценарий AI моделът не разчита на динамично търсене в интернет при всяко запитване, а работи върху предварително обхванат и ясно дефиниран набор от страници. Това осигурява контролируем обхват на анализа и елиминира зависимостта от алгоритми за външно търсене.

Въпреки техническата възможност, към момента на пазара почти не съществуват публично достъпни системи, които целенасочено обединяват автоматизирано обхождане на конкретен сайт, структурирано индексирание на съдържанието и интегриран езиков модел за проследим и доказуем анализ. Обикновено crawler инструментите и AI моделите се разглеждат като отделни категории технологии. В единия случай се събира информация без интелигентен интерфейс, а в другия се генерират отговори без гарантиран контрол върху обхвата на използваното съдържание.

Поради това класическите scraping и crawling технологии могат да бъдат определени като подценен инструмент в контекста на интелигентното уеб проучване. Те притежават механизма за систематично и пълно събиране на данни, който липсва при стандартните AI системи. В комбинация с езиков модел и подходящ механизъм за индексирание те могат да се превърнат в основа за изграждане на контролируема и проверима изследователска среда, в която всяко заключение е обвързано с предварително събрана и ясно дефинирана информационна база.

Именно тази идея за обединяване на систематично обхождане на уеб съдържание и интелигентен езиков анализ стои в основата на разработваната Scholia. Тя цели да съчетае контролиран и изчерпателен сбор на информация с



възможност за семантично търсене и проследим отговор, изграден върху ясно дефиниран и предварително обхванат набор от източници.

1.3. Извод от прегледа: какви пропуски остават и как Scholia ги попълва

От направения преглед се вижда, че съществуващите решения оптимизират различни елементи от процеса на уеб проучване, но не обединяват всички необходими характеристики в една система. Част от тях улесняват достъпа до информация чрез синтез, други подобряват прозрачността чрез цитиране, трети осигуряват контрол върху предварително зададен набор от текстове, а класическите crawler инструменти гарантират систематично събиране на съдържание. Липсва обаче решение, което едновременно да гарантира фиксиран и измерим обхват на изследване и да осигурява пряка връзка между всяко твърдение и конкретен текстов откъс.

Дипломната работа се позиционира именно в тази празнина. Scholia комбинира автоматизирано обхождане на уеб съдържание с механизъм за семантично търсене и доказателствено обвързване на отговорите. По този начин тя адресира нуждата от инструмент, който не само синтезира информация, но и работи в рамките на ясно дефиниран обхват и предоставя проверима основа за всяко генерирано заключение.



Втора глава: Изисквания и проектиране

2.1. Функционални изисквания

2.1.1. Потребителска автентикация и изолация на данни

Системата трябва да позволява регистрация и вход чрез електронна поща и парола. Данните на всеки потребител (разговори, източници, индексирано съдържание) трябва да бъдат логически изолирани от тези на останалите потребители.

2.1.2. Създаване и конфигуриране на източник (Source)

Потребителят трябва да може да създаде нов източник, като зададе начална страница и режим на обхождане. Системата трябва да поддържа статични режими с фиксиран лимит на страници и динамични режими с възможност за последващо разширяване на корпуса.

2.1.3. Автоматизирано обхождане на уеб страници

При добавяне на източник системата трябва автоматично да обхожда страниците в рамките на зададения обхват, да извлича текстово съдържание и да съхранява връзките между страниците.

2.1.4. Индексиране и семантично представяне на съдържание

Системата трябва да разделя извлеченото съдържание на текстови сегменти и да създава векторни представления (embeddings), които позволяват последващо семантично търсене.

2.1.5. Задаване на въпроси върху избран корпус

Потребителят трябва да може да задава въпроси върху избрания източник. Системата трябва да извлича релевантни текстови откъси и да генерира отговор, базиран единствено на индексираното съдържание.

2.1.6. Генериране на отговор с цитати

Всеки генериран отговор трябва да съдържа препратки към конкретни текстови откъси и страници, които служат като доказателствена основа на твърденията.

2.1.7. Динамично разширяване на корпуса



В динамичен режим системата трябва да може да предлага нови страници за добавяне към корпуса, когато наличната информация е недостатъчна за пълен отговор.

2.1.8. Визуализация на граф и прогрес

Системата трябва да визуализира връзките между страниците под формата на граф и да показва в реално време процеса на обхождане.

2.2. Нефункционални изисквания

2.2.1. Проследимост и проверимост

Всяко твърдение в генерирания отговор трябва да бъде обвързано с конкретен текстов откъс от индексирания източник.

2.2.2. Контролиран обхват на анализа

Системата трябва да работи в рамките на ясно дефиниран корпус от страници, определен от потребителя чрез избрания източник и режим на обхождане.

2.2.3. Спазване на уеб стандарти и етика

Обхождането трябва да уважава ограниченията, зададени в robots.txt файла на съответния сайт и да използва коректна идентификация чрез User-Agent.

2.2.4. Ефективност при семантично търсене

Търсенето на релевантни текстови сегменти трябва да се извършва чрез индексирание, което позволява бърза обработка при нарастващ обем от данни.

2.2.5. Реално време и синхронизация

Промените в процеса на обхождане и индексирание трябва да се отразяват в интерфейса в реално време.

2.3. Алгоритъм и архитектура: резюмирани

Алгоритъмът на системата Scholia представлява последователна комбинация от автоматизирано събиране на уеб съдържание, семантично индексирание и многостепенно разсъждение върху предварително дефиниран корпус от текстове. Основната му цел не е единствено генериране на граматически коректен отговор, а изграждане на контролируема изследователска среда, в която обхватът на анализа е ясно определен, а всяко твърдение може да бъде проследено до конкретен текстов откъс. Поради това архитектурата разделя



процеса на няколко логически етапа – обхождане, индексирание, извличане и структурирано разсъждение, които функционират самостоятелно, но са свързани в общ алгоритмичен поток.

Първият етап е обхождането на уеб съдържание. За всеки добавен източник се създава отделна задача (crawl job), която се изпълнява от работен процес. Придвижването през страниците се реализира чрез обхождане в ширина (Breadth-First Search), което позволява систематично преминаване през вътрешната структура на сайта слой по слой, започвайки от началния URL. Поддържа се опашка от адреси за посещение, както и множество от вече обработени страници, за да се избегнат повторения. При всяка итерация се извлича следващият URL, проверява се дали достъпът е разрешен според robots.txt файла на съответния сайт, изтегля се HTML съдържанието и се отделя основния текстов фрагмент. След това се извличат вътрешните връзки, които преминават през нормализация и филтриране с цел премахване на дублирания, външни домейни и нежелани ресурси. Връзките между страниците се записват като ребра в графова структура, която моделира топологията на изследвания домейн. Така се изгражда фиксиран корпус от страници, върху който по-късно се извършва семантичният анализ.

Освен статичните режими на обхождане, при които предварително е зададен максимален брой страници (напр. 1, 5, 15 или 35), системата поддържа и динамичен режим. При него първоначалното обхождане е ограничено, но впоследствие корпусът може да бъде разширяван в зависимост от конкретните въпроси на потребителя. По време на обхождането всяка открита, но неизтеглена страница може да бъде семантично кодирана. В режима dynamic surface се съхранява контекста около самата връзка в изходната страница, докато в режима dynamic dive се извлича начален откъс от съдържанието на целевата страница. Тези данни се индексират и впоследствие служат за интелигентен избор на следваща страница за добавяне към корпуса. Ако в процеса на разсъждение се установи липса на достатъчно доказателства, системата може да предложи разширяване на корпуса чрез добавяне на нова страница, която е семантично най-релевантна спрямо текущата подзаявка. Така динамичният режим позволява адаптивно разширяване на обхвата без да се нарушава контролът върху използваните източници.



След приключване на първоначалното обхождане започва етапът на индексирание. Съдържанието на всяка страница се разделя на по-малки текстови сегменти (chunks), които имат ограничена дължина и частично припокриване помежду си. Този подход запазва смисловата цялост на текста, като същевременно осигурява достатъчно контекст за надеждно семантично представяне. Всеки сегмент се преобразува чрез embedding модел в числов вектор, който описва неговото съдържание в многомерно пространство. Векторите се съхраняват в база данни, разширена с pgvector, което позволява изчисляване на семантична близост чрез косинусова дистанция. За оптимизация на търсенето се използва HNSW индекс, който позволява бързо приближено намиране на най-близките вектори дори при по-голям обем данни.

При задаване на въпрос системата преминава към етап на извличане. Потребителската заявка първо се преобразува в embedding, след което се извършва similarity търсене спрямо всички индексирани сегменти, принадлежащи към съответния разговор. Извличат се най-релевантните текстови откъси, които служат като доказателствен материал. Когато въпросът е комплексен, той се разлага на няколко подзаявки, а резултатите се разпределят чрез механизъм за балансирано покритие, така че нито една подзаявка да не доминира изцяло контекстния прозорец на модела.

Същинската отличителна част на алгоритъма е многостепенният процес на разсъждение. Вместо директно генериране на текстов отговор, въпросът преминава през планираща фаза, в която се идентифицират структурирани информационни единици, наречени слотове. Те могат да бъдат скаларни (единична стойност), списъчни (множество стойности) или от тип съпоставяне (mapping), при което за всеки ключ трябва да бъде намерена съответна стойност. Слотовете позволяват формализиране на логическата структура на отговора и ясно разграничаване между отделните компоненти на търсената информация. Възможни са зависимости между слотовете, което дава възможност за поетапно разсъждение, при което попълването на един слот служи като основа за следващия.

След планирането започва итеративен цикъл, който включва последователно извличане на доказателства, структурирано извличане на твърдения и оценка на степента на попълване на слотовете. При всяка итерация



системата изпълнява RAG заявки, анализира върнатите сегменти и извлича конкретни твърдения, които се записват като структурирани стойности в базата данни. Всяка стойност се свързва с поне един доказателствен откъс. След това се извършва оценка на пълнотата – например дали при списъчен слот е достигнат зададеният брой елементи или дали при mapping слот са намерени стойности за всички ключове. Ако системата работи в динамичен режим и наличните доказателства са недостатъчни, в рамките на същия цикъл може да бъде предложено действие за разширяване на корпуса чрез добавяне на нова страница.

Процесът продължава до достигане на достатъчна пълнота или до установяване на застой, при който не се добавя нова релевантна информация. В заключителната фаза се генерира финален отговор въз основа на всички акумулирани доказателствени сегменти. Моделът формулира текста, като маркира твърденията с референции към конкретни източници. Само изреченията, които действително подкрепят съответното твърдение, се включват като цитати и се съхраняват отделно с връзка към оригиналната страница и текстов фрагмент.

В обобщение, алгоритъмът на Scholia интегрира графово обхождане, векторно индексирание и контролируем процес на разсъждение в единна система. Статичните режими осигуряват предвидим и ограничен обхват, докато динамичният режим позволява адаптивно разширяване на корпуса при необходимост. Чрез структуриране на знанието в слотове и обвързване на всяко твърдение с конкретен доказателствен откъс, системата гарантира по-висока степен на прозрачност и проверимост в сравнение със стандартните езикови модели, които работят върху неконтролируем или частичен информационен контекст.

2.4. Технологии - език, framework, БД, векторно хранилище

2.4.1. React + TypeScript + Vite

React е избран като основна библиотека за изграждане на потребителския интерфейс. Имам предходен практически опит с него, което позволи по-бързо и уверено реализиране на по-сложните части на системата. По-съществената причина за избора му обаче е компонентният модел. Интерфейсът не е просто статична страница, а среда с множество взаимодействащи елементи – чат, панели със страници, модални прозорци, визуализации. Подходът с компоненти позволява всяка функционална част да бъде изградена като самостоятелна



единица с ясна логика и отговорности. В сравнение с класическа структура от отделни HTML, CSS и JavaScript файлове, този модел дава по-добра организация на кода, по-лесна поддръжка и възможност за постепенно разширяване. Допълнително предимство е добрата интеграция с използваните библиотеки за заявки и визуализация (например TanStackQuery, D3.js), което улеснява изграждането на по-динамични елементи в интерфейса.

Заедно с React е използван TypeScript като основен език за клиентската част. Макар JavaScript да е напълно функционален, TypeScript добавя статична типизация, която значително намалява риска от логически грешки. В приложението се работи с различни структури от данни – разговори, съобщения, страници, текстови сегменти и цитати – и при липса на типова проверка несъответствията често се откриват едва при изпълнение на кода. TypeScript позволява подобни проблеми да бъдат засечени още на етап разработка. Ясното дефиниране на типовете за API отговори и вътрешни обекти прави кода по-надежден и по-лесен за надграждане.

Vite е използван като инструмент за разработка и изграждане на фронтенд частта на приложението поради своята бързина и минималистична конфигурация. Той стартира проекта почти мигновено и осигурява изключително бързо отразяване на промени в кода, което е особено важно при интензивни итерации върху интерфейса и потребителското изживяване. Благодарение на Hot Module Replacement (HMR), промените в отделни компоненти се прилагат без презареждане на цялото приложение, което ускорява разработката и намалява прекъсванията в работния процес. Освен това Vite използва съвременния ES module базиран подход и оптимизира зависимостите по време на разработка, което допринася за по-добра производителност и по-чист build процес.

2.4.2. Tailwind CSS + ShadCN / Radix UI

За изграждането на интерфейса е използван Tailwind CSS за стилизиране и подредба на елементите. Той позволява бързо оформяне на layout-a, бутоните, картите и чат съобщенията директно в компонентите, без нужда от отделни CSS файлове. Това прави кода по-прегледен и улеснява промени по дизайна.

Shadcn/ui е използван като библиотека от готови React компоненти, изградени върху Radix UI. Тя предоставя предварително структурирани елементи като диалози, dropdown менюта, табове, бутони и форми, които могат да бъдат



персонализирани чрез Tailwind. За разлика от традиционни UI библиотеки, shadcn/ui не налага собствен визуален стил, а служи като основа, която разработчикът може свободно да адаптира според нуждите на проекта. Radix UI стои в основата на тези компоненти и осигурява стабилно поведение и коректно управление на фокус и клавиатурна навигация.

2.4.3. Supabase (PostgreSQL + Auth + Realtime + Edge Functions)

Supabase е избран като основна backend платформа, защото предоставя завършена среда за разработка, която значително намалява инфраструктурната сложност. Вместо отделно конфигуриране на база данни, authentication слой, realtime комуникация и сървърна логика, Supabase обединява тези функционалности в единна система с ясен и последователен модел на работа. Това позволява фокусът да бъде върху логиката на приложението, а не върху поддръжка и интеграция на множество отделни услуги.

В рамките на проекта най-активно са използвани три основни възможности на Supabase: базата данни, Realtime функционалността и Edge Functions. Базата данни служи като single source of truth за разговорите, страниците, текстовите сегменти и процесите по обхождане. Работата с нея е улеснена от автоматично генерирания API и ясният модел за достъп до данните, което прави разработката значително по-бърза в сравнение с директна работа с „raw“ PostgreSQL и собствен backend слой.

Realtime функционалността е една от ключовите причини за избора на Supabase. Тя позволява клиентската част да получава автоматични обновявания при промяна на данни в базата, без нужда от ръчно имплементиране на WebSocket сървър или polling логика. Това е особено полезно при процеси като обхождане и индексирание, където статусът и прогресът трябва да се визуализират в реално време. Реализацията на този механизъм е директна и стабилна, тъй като е интегрирана на ниво база данни.

Edge Functions са използвани за изпълнение на сървърна логика, която не е подходящо да се намира в клиентската част. Чрез тях в Scholia се реализират операции като обработка на RAG заявки и добавяне на нови страници за индексирание. Изпълнението на тази логика извън браузъра позволява сигурно използване на външни API услуги и по-добър контрол върху обработката на данни, без необходимост от поддръжане на собствен сървър.



Допълнително предимство на Supabase е ниският праг за разработка и добрата документация. Позволява бързо изграждане на работещ backend без сложна първоначална конфигурация, което я прави особено подходяща за проекти с ограничено време за реализация. В сравнение с алтернативи, които изискват отделно управление на база данни, authentication и serverless функции, Supabase предлага по-опростен и директен работен процес.

Поради тези причини Supabase е предпочетен като основа за backend логиката на системата.

2.4.4. TanStack Query

TanStack Query е библиотека за управление на сървърно състояние в клиентски приложения и в проекта се използва за управление на асинхронните заявки и синхронизацията на данните в клиентската част. Приложението работи с динамична информация като разговори, съобщения, страници и crawling процеси, които се променят във времето и трябва да се отразяват коректно в интерфейса. Вместо ръчно обновяване на състоянието след всяка заявка, TanStack Query поема кеширането, повторното извличане и обновяването на данните при настъпила промяна. В комбинация с Realtime механизма на Supabase това позволява интерфейсът да се актуализира автоматично при ново съобщение или промяна в статус на обхождане. Използването на TanStack Query прави логиката по-структурирана и намалява риска от разминаване между визуализираното и действителното състояние на данните.

2.4.5. Pgvector

Pgvector е разширение към PostgreSQL, което позволява в базата данни да се съхраняват числови представяния на текст. Тъй като Scholia използва RAG (Retrieval-Augmented Generation), за да намери точни цитати в извлечените данни, pgvector идва на помощ за поддържането на база от embedded вектори, които заместват chunk-овете, на които е разделено индексираният съдържание на уеб страниците. Когато даден текст се преобразува чрез embedding модел, той се превръща в списък от числа, който описва смисъла му. Pgvector позволява тези списъци от числа да се записват в таблиците и да се сравняват помежду си.

2.4.6. OpenAI - text-embedding-3-small, gpt-4o-mini



В системата се използват два различни модела на OpenAI с ясно разграничени функции. Text-embedding-3-small служи за преобразуване на текст в числово представяне. Той се използва при индексирание на съдържание от обхожданите страници и при обработка на потребителските въпроси. Всеки текстов сегмент се преобразува в embedding, който се съхранява в базата, а при задаване на въпрос той също се преобразува в embedding и се сравнява с наличните. Така се откриват най-близките по смисъл откъси. Този модел е избран, защото осигурява достатъчна точност за семантично търсене при по-ниска цена и по-бърза обработка в сравнение с по-големи модели (за задачата не ни трябват големите възможности, които алтернативата text-embedding-3-large предлага).

Gpt-4o-mini се използва за генериране на отговорите в чата. След като бъдат избрани релевантните текстови откъси, моделът формулира финалния отговор въз основа на тях. Освен това се използва за разделяне на по-сложни въпроси на подчасти и за извличане на структурирана информация от въпросите на потребителя. Той е за предпочитане пред по-тежки модели, тъй като предлага добро качество на текста при по-нисък разход и по-бързо време за отговор, което е подходящо за Scholia.

2.4.7. Cheerio + node-fetch + robots-parser

За реализиране на процеса по обхождане на уеб страници са използвани библиотеките node-fetch, Cheerio и robots-parser. Тези инструменти позволяват автоматично изтегляне, обработка и анализ на HTML съдържание, без необходимост от тежки браузърни решения.

Node-fetch се използва за изпращане на HTTP заявки към даден URL и получаване на неговото съдържание. Той представлява сървърна реализация на стандартния fetch механизъм и позволява контролирано извличане на HTML от страниците, които системата обхожда.

Cheerio служи за parse-ването на полученото HTML съдържание. Той позволява достъп до елементите на страницата чрез синтаксис, подобен на jQuery, което улеснява извличането на основния текст, линковете и други необходими елементи. Чрез него се реализира отделянето на съдържанието от навигация, скриптове и други несъществени части.



Robots-parser се използва, за да се провери дали даден URL е позволен за обхождане според файла robots.txt на съответния сайт. Това гарантира, че процесът по индексирание спазва правилата, зададени от администратора на сайта, и следва добрите практики за етично обхождане.

Комбинацията от тези три библиотеки позволява контролиран crawler механизъм, който извлича текстово съдържание, следва вътрешни връзки и спазва ограниченията на сайтовете, без излишна сложност.

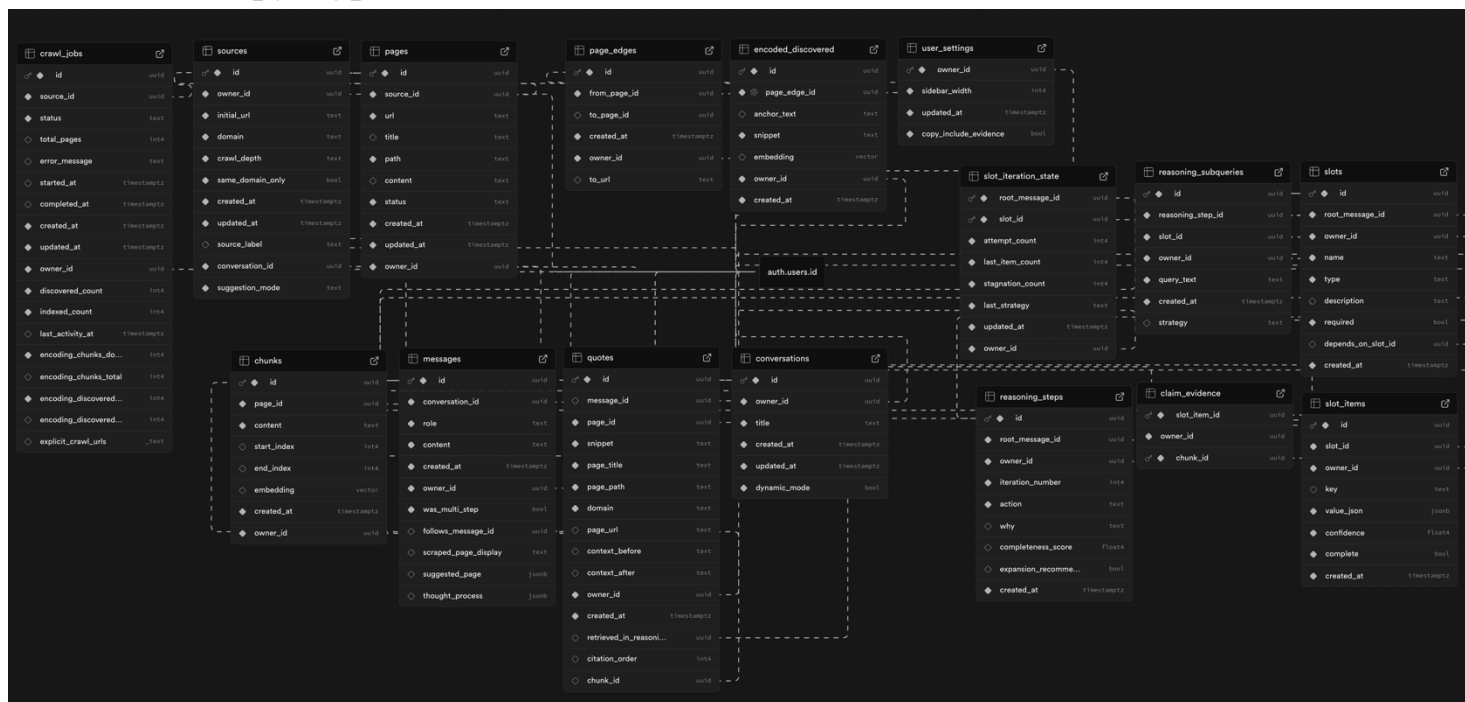
2.4.8. D3.js

D3.js е използвана за визуализация на връзките между индексираните страници под формата на граф. Чрез нея се изгражда интерактивна структура от възли и връзки, която показва как отделните страници са свързани помежду си чрез линкове. Библиотеката позволява динамично позициониране, мащабиране и преместване на елементите, което прави графа удобен за разглеждане дори при по-голям брой страници. Избрах D3 поради високото ниво на персонализация, вместо използване на готови, ограничени графични решения.

2.4.9. Vercel

За deployment на приложението се използва Vercel. Платформата предоставя автоматизиран процес по изграждане и публикуване на всяка нова версия чрез интеграция с Git репозитория. При всяко commit-ване към основния клон се стартира автоматичен build, което позволява бърза итерация и лесно управление на версиите. Vercel осигурява и оптимизации като CDN разпространение, автоматично управление на environment променливи и preview среди за всеки pull request, което значително улеснява тестването и поддръжката на приложението в продукционна среда.

2.5. Структура на базата данни



Фиг. 2.5.1

Следните таблици (Фиг. 2.5.1) съществуват за всеки потребител и **всяка** от тях съдържа **ненулево поле owner_id**. Въпреки че това не представлява единствен източник на истина (single source of truth), решението е умишлено. При multi-tenant SaaS приложения, такива в които данните на всеки потребител са логически изолирани от тези на останалите, е популярна практика във всяка таблица да присъства owner_id или organization_id [2.5.1 - 1]. Този подход реализира модел на per-user data ownership и позволява дефиниране на Row Level Security (RLS), съществуващи при всяка таблица, да е лесно за поддръжка (ясни заявки без хиляди JOIN-ове).

Заради многото връзки е трудно да се подредят в положение, в което ясно се виждат на диаграмата, затова точките по-долу обясняват релациите в базата в повече детайли.

2.5.1. Таблица за потребителски настройки



user_settings	
owner_id	uuid
sidebar_width	int4
updated_at	timestampz
copy_include_evidence	bool
suggested_page_can...	int4

Фиг. 2.5.1.1

Таблицата `user_settings` съхранява (Фиг. 2.5.1.1) потребителски конфигурируеми настройки, като широчината на левия сайдбар и предпочитанието дали при копиране на отговорите да се включват или изключват доказателствените цитати.

2.5.2. Таблици, свързани с цитираните източници - sources, pages & crawl jobs

crawl_jobs	sources	pages
id (uuid)	id (uuid)	id (uuid)
source_id (uuid)	owner_id (uuid)	source_id (uuid)
status (text)	initial_url (text)	url (text)
total_pages (int4)	domain (text)	title (text)
error_message (text)	crawl_depth (text)	path (text)
started_at (timestampz)	same_domain_only (bool)	content (text)
completed_at (timestampz)	created_at (timestampz)	status (text)
created_at (timestampz)	updated_at (timestampz)	created_at (timestampz)
updated_at (timestampz)	source_label (text)	updated_at (timestampz)
owner_id (uuid)	conversation_id (uuid)	owner_id (uuid)
discovered_count (int4)	suggestion_mode (text)	
indexed_count (int4)		
last_activity_at (timestampz)		
encoding_chunks_discovered (int4)		
encoding_chunks_total (int4)		
encoding_discovered (int4)		
encoding_discovered (int4)		
explicit_crawl_urls (_text)		

Фиг. 2.5.2.1

Таблиците sources, pages и crawl_jobs (Фиг. 2.5.2.1) изпълняват различни, макар и свързани роли в архитектурата на системата. Един source представлява логически източник на информация – група от уеб страници, които се третират като общ корпус от знания. При добавяне на source се конфигурират параметри като начална страница, дълбочина на обхождане и ограничения по домейн. Всяка реално извлечена (scraped) страница се съхранява като запис в таблицата pages. Между sources и pages съществува релация one-to-many, реализирана чрез поле source_id в таблицата pages.

Таблицата crawl_jobs моделира конкретен период от време, в който crawler-ът е активен. Всеки crawl job представлява отделна сесия на обхождане, свързана

с конкретен source. При първоначално добавяне на source се създава crawl job, но такива могат да възникват и впоследствие – например при повторно обхождане (recrawl) или при добавяне на нова предложена страница към динамичен source. Поради това между sources и crawl_jobs също съществува релация one-to-many.

2.5.3. Таблица, свързана с RAG - Chunks

След извличане съдържанието на всяка страница се разделя на по-малки текстови сегменти (chunks), които се съхраняват в таблицата chunks (Фиг. 2.5.3.1). Между pages и chunks съществува релация one-to-many, тъй като всяка страница може да бъде разбита на множество сегменти. Всеки chunk съдържа текстов откъс, позиционна информация и съответния векторен embedding, използван за семантично търсене и retrieval процеси.

chunks		
 	id	uuid
	page_id	uuid
	content	text
	start_index	int4
	end_index	int4
	embedding	vector
	created_at	timestamptz
	owner_id	uuid

Фиг. 2.5.3.1

2.5.4. Таблица, свързана с графа - Page Edges

Връзките между страниците се моделират чрез таблицата page_edges (Фиг. 2.5.4.1). Един edge може да представлява връзка между вече индексирани страници (page-to-page), както и връзка от страница към URL, който все още не е обходен. Във втория случай се моделира потенциална бъдеща връзка, която може да бъде реализирана при последващо обхождане. При динамични sources за всеки нереализиран линк се създава запис в таблицата encoded_discovered, където се съхранява embedding на контекста около линка или на началния текст на целевата

страница, в зависимост от режима на работа (surface vs dive). Тези embeddings се използват за семантично предложение на нови страници чрез RAG механизъм.

page_edges		
id	uuid	
from_page_id	uuid	
to_page_id	uuid	
created_at	timestampz	
owner_id	uuid	
to_url	text	

Фиг. 2.5.4.1

2.5.5. Таблицы свързани с чат – conversations, messages и quotes

conversations	messages	quotes
id	id	id
owner_id	conversation_id	message_id
title	role	page_id
created_at	content	snippet
updated_at	created_at	page_title
dynamic_mode	owner_id	page_path
	was_multi_step	domain
	follows_message_id	page_url
	scraped_page_display	context_before
	suggested_page	context_after
	thought_process	owner_id
		created_at
		retrieved_in_reasoni...
		citation_order
		chunk_id

Фиг. 2.5.5.1

Таблицата conversations (Фиг. 2.5.5.1) съхранява отделните продължителни чат сесии, в които участва потребителят. Между conversations и messages

съществува релация one-to-many, като messages съдържа както въпросите на потребителя, така и отговорите на модела, разграничени чрез поле role. Всеки assistant отговор обикновено се подкрепя от група цитати, съхранявани в таблицата quotes. Между messages и quotes съществува релация one-to-many, като всеки quote е свързан чрез page_id с конкретната страница, от която произлиза текстовият откъс. По този начин се осигурява проследимост на всяко твърдение до оригиналния източник.

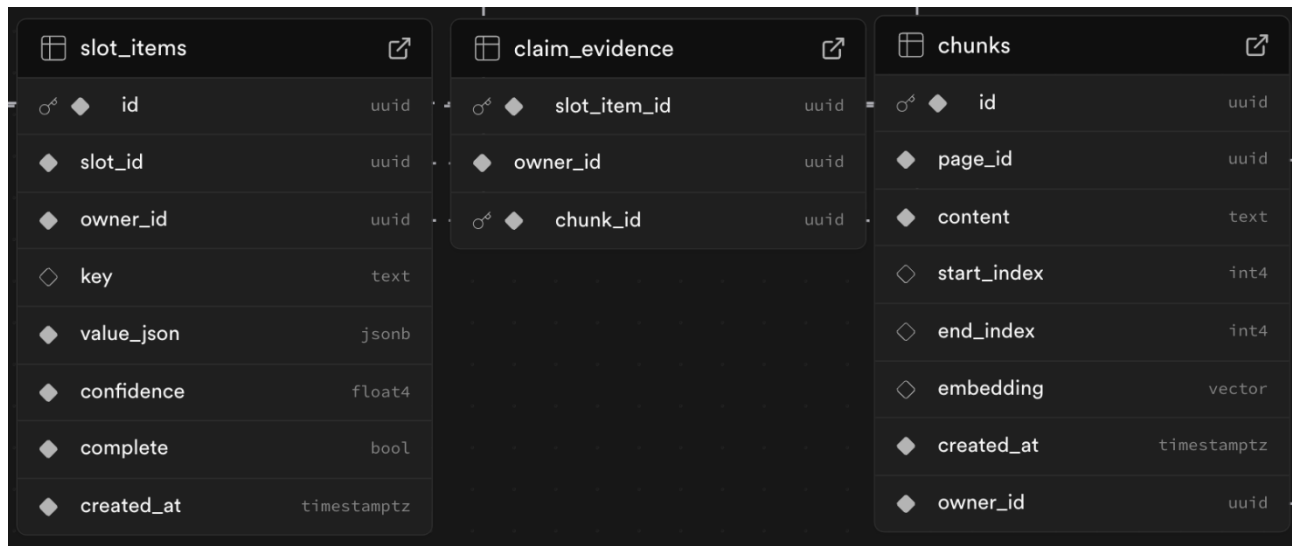
2.5.6. Таблици, свързани с процеса на разсъждение на модела – reasoning_steps, slots, slot_items, claim_evidence и reasoning_subqueries

reasoning_steps	slots	slot_items
id (uuid)	id (uuid)	id (uuid)
root_message_id (uuid)	root_message_id (uuid)	slot_id (uuid)
owner_id (uuid)	owner_id (uuid)	owner_id (uuid)
iteration_number (int4)	name (text)	key (text)
action (text)	type (text)	value_json (jsonb)
why (text)	description (text)	confidence (float4)
completeness_score (float4)	required (bool)	complete (bool)
expansion_recomme... (bool)	depends_on_slot_id (uuid)	created_at (timestampz)
created_at (timestampz)	created_at (timestampz)	

Фиг. 2.5.6.1

Системата използва и слой за структурирано разсъждение. Всеки assistant message може да бъде разделен на логически стъпки, които се съхраняват в таблицата reasoning_steps (Фиг. 2.5.6.1). Тези стъпки моделират процеса на анализ и позволяват проследяване на итеративното изграждане на отговора. В рамките на този процес се използват междинни променливи, съхранявани в таблицата slots. Всеки slot има име, описание и тип – скаларна стойност, списък или речник. Слотите от тип „речник“ (напр. „biggest accomplishment of each of Obama’s children“) могат да зависят от друг слот – в случая списъкът „Obama’s children“, който трябва първо да бъде попълнен; затова е предвидена self-релация чрез полето depends_on_slot_id. Конкретните стойности на тези слот променливи се

съхраняват в таблицата `slot_items` (Фиг. 2.5.6.2), която е в релация `many-to-one` със `slots`.



Фиг. 2.5.6.2

Всеки `slot item` трябва да бъде подкрепен с доказателство от индексирани източници. Поради това между `slot_items` и `chunks` съществува `many-to-many` връзка, реализирана чрез междинната таблица `claim_evidence`. Тази структура гарантира, че всяко извлечено твърдение или междинно заключение в процеса на разсъждение е обвързано с конкретен текстов фрагмент от източник.



reasoning_subqueries	slots	reasoning_steps
- id (uuid) - reasoning_step_id (uuid) - slot_id (uuid) - owner_id (uuid) - query_text (text) - created_at (timestampz) - strategy (text)	- id (uuid) - root_message_id (uuid) - owner_id (uuid) - name (text) - type (text) - description (text) - required (bool) - depends_on_slot_id (uuid) - created_at (timestampz) - target_item_count (int4) - current_item_count (int4) - attempt_count (int4) - finished_querying (bool) - last_queries (_text) - items_per_key (int4)	- id (uuid) - root_message_id (uuid) - owner_id (uuid) - iteration_number (int4) - action (text) - why (text) - completeness_score (float4) - expansion_recommen... (bool) - created_at (timestampz)

Фиг. 2.5.6.3

Допълнително, таблицата `reasoning_subqueries` (Фиг. 2.5.6.3) съхранява оптимизирани подзаявки, генерирани от модела по време на разсъждението. Те са свързани както с конкретен `reasoning step`, така и с конкретен `slot`, и служат за фокусирано извличане на релевантна информация от индексирания корпус чрез RAG. По този начин системата комбинира графова структура на знанията, семантично индексирание и контролируем процес на разсъждение, при който всяко твърдение остава проследимо до първичен източник.

Трета глава: Реализация на системата Scholia

Главата представя практическата реализация на системата Scholia и описва конкретните алгоритми и механизми, чрез които теоретичният модел от предходните раздели е имплементиран в работещо уеб приложение. В нея последователно се разглеждат процесите по обхождане на уеб съдържание, индексирание и семантично търсене чрез RAG механизъм, динамично разширяване на корпуса, както и многостепенния алгоритъм за разсъждение и генериране на аргументиран отговор. Целта на тази глава е да демонстрира как архитектурните решения и поставените изисквания се реализират на алгоритмично и системно ниво.

3.1. Обхождане на сайтове

За да може платформата да отговаря на въпроси на база избрани източници (например статии от Wikipedia), е необходимо предварително събиране и индексирание на тяхното съдържание. Този процес започва с обхождане (crawling) – автоматизирано посещаване на начална страница, извличане на съдържанието ѝ и последователно преминаване през свързаните с нея страници в рамките на зададен обхват. В резултат се изгражда насочен граф от страници и връзки между тях, който впоследствие се използва както за семантично търсене, така и като контекст при генериране на отговори.

В настоящата секция се описва архитектурата на този процес: как се създават и управляват задачите за обхождане, по какъв алгоритъм се избира следващата страница за посещение, как се съхраняват страниците и връзките между тях, как се спазват ограниченията на сайтовете (robots.txt), както и как се филтрират препратките с цел изграждане на смислен и съдържателен граф.

Обхождането е организирано като задачи (jobs), които биват терминирани ако надвишат 5 минути. За всеки източник (source - логическа група от взаимосвързани страници) се създава отделна задача. Работник (worker) започва от началния URL адрес и извършва обхождане по алгоритъма BFS (Breadth-First Search), използвайки опашка от URL адреси за посещение. На всяка стъпка се зарежда следващата страница, извличат се препратките, записва се съдържанието ѝ, както и ребрата в графа (връзките от текущата страница към всеки открит URL адрес). Новооткритите URL адреси се добавят в опашката, докато не бъде

достигнат лимита за брой страници (определен от дълбочината на обхождане) или опашката не се изчерпи.

След приключване на обхождането се стартира RAG (Retrieval-Augmented Generation) индексирание върху страниците от съответния разговор, което подготвя данните за семантично търсене и последващо генериране на аргументирани отговори.

3.1.1. Robots Parser

При уеб обхождане е необходимо да се спазват правилата, зададени от собствениците на сайтове. За тази цел се използва стандарта robots.txt – текстов файл, разположен в основната директория на сайта (напр. <https://example.com/robots.txt>), който указва кои части от сайта могат или не могат да бъдат обхождани от автоматизирани агенти (crawlers). Файлът съдържа правила за различни потребителски агенти (User-agents) и позволява ограничаване на достъпа до определени пътища, с цел защита на чувствителни ресурси, намаляване на натоварването на сървъра и спазване на добри практики в уеб екосистемата (Фиг. 3.1.1.1).

```
const firstSeedUrl = seedUrls[0];
let robotsParser: ReturnType<typeof RobotsParser> | null = null;
try {
  const robotsUrl = new URL('/robots.txt', firstSeedUrl).toString();
  const robotsResponse = await fetch(robotsUrl);
  if (robotsResponse.ok) {
    const robotsText = await robotsResponse.text();
    robotsParser = RobotsParser(robotsUrl, robotsText);
  }
} catch (err) {
  console.warn('crawl: robots.txt unavailable', firstSeedUrl.slice(0, 50), err instanceof Error
    ? err.message : err);
}
```

Фиг. 3.1.1.1

В реализираната система при стартиране на обхождането се прави опит за изтегляне и анализ на robots.txt от съответния сайт. Ако файлът е наличен, за всяка страница се проверява дали достъпът е разрешен за използвания потребителски агент, в този случай нашия „ScholiaCrawler“.

```
if (robotsParser && !robotsParser.isAllowed(normalizedUrl, 'ScholiaCrawler')) {
  continue;
}
```

Фиг. 3.1.1.2

Ако даден URL е забранен според правилата, той не се включва в обхождането (Фиг. 3.1.1.2). По този начин се гарантира, че системата уважава ограниченията, зададени от собствениците на сайтовете.

3.1.2. Алгоритъм за обхождане на източници чрез BFS (скелет)

```
const visited = new Set<string>();
const discovered = new Set<string>();
const queue: string[] = [...seedUrls];
seedUrls.forEach((u) => discovered.add(u));
```

Фиг. 3.1.2.1

Създаваме опашка queue и пазим два сета: visited за посетени и напълно изтеглени сайтове и discovered за линкове, които сме открили в съдържанието на сайтовете (Фиг. 3.1.2.2).

```
while (queue.length > 0 && visited.size < maxPages) {
  const url = queue.shift()!;

  const result = await crawlPage(normalizedUrl, source, conversationId);
  if (!result) {
    visited.add(normalizedUrl);
    continue;
  }

  const { page, html } = result;
  visited.add(normalizedUrl);
```

Фиг. 3.1.2.2

При всяка итерация се обхожда URL адресът, който се намира най-отпред на опашката (First In First Out).

```
const isDynamic = source.crawl_depth === 'dynamic';
const links = extractLinks(html, normalizedUrl, source);
const linksWithContext = isDynamic ? extractLinksWithContext(html, normalizedUrl, source) : [];

const edgesToInsert: Array<{ from_page_id: string; to_url: string; owner_id: string }> = [];
const linksToProcess = isDynamic ? links.slice(0, MAX_LINKS_PER_PAGE_DYNAMIC) : links;
```

Фиг. 3.1.2.3

Чрез функцията `extractLinks` (Фиг. 3.1.2.3) се извличат всички препратки, свързани с текущата страница. При динамичен режим е въведен лимит `MAX_LINKS_PER_PAGE_DYNAMIC`, зададен на 200, тъй като за всяка открита връзка се кодира и контекста около нея. Това ограничава броя обработвани линкове с цел контрол върху изразходваните токени и ресурсите, използвани за `embedding` модела. При статичния режим такъв лимит не е необходим, тъй като не се извършва контекстно кодиране на всяка препратка (Фиг. 3.1.2.4).

```
for (const link of linksToProcess) {
  edgesToInsert.push({
    from_page_id: page.id,
    to_url: normalizedLink,
    owner_id: source.owner_id,
  });
}
```

Фиг. 3.1.2.4

Всеки открит `edge` се добавя в масив, който впоследствие се записва в таблицата `page_edges` в базата данни (Фиг. 3.1.2.5). По този начин се изгражда графовата структура на връзките между страниците, която впоследствие се визуализира в интерфейса (графът в долния ляв панел).

```
if (!discovered.has(normalizedLink) && !visited.has(normalizedLink)) {
  discovered.add(normalizedLink);
  queue.push(normalizedLink);
}
```

Фиг. 3.1.2.5

Всички новооткрити линкове се добавят в края на опашката, ако вече не са обходени или добавени (Фиг. 3.1.2.6). По този начин се реализира обхождане в ширина: първо се обработват началните URL адреси, след това страниците,

директно свързани с тях, после страниците на следващото ниво на свързаност и т.н. Така графът се обхожда слой по слой според дълбочината.

```
if (edgesToInsert.length > 0) {
  const batchSize = 50;
  for (let i = 0; i < edgesToInsert.length; i += batchSize) {
    const chunk = edgesToInsert.slice(i, i + batchSize);
    const { error: edgeErr } = await supabase
      .from('page_edges')
      .upsert(chunk, { onConflict: 'from_page_id,to_url', ignoreDuplicates: true });
    if (edgeErr) {
      console.error('crawl: edge insert failed', edgeErr.message);
    }
    if (i + batchSize < edgesToInsert.length) {
      await new Promise((resolve) => setTimeout(resolve, 10));
    }
  }
}
```

Фиг. 3.1.2.6

Тук откритите `page_edges` се записват в базата данни поетапно, на партии от по 50 записа. Този batch-подход намалява натоварването върху базата и предотвратява достигане на rate limit при голям брой едновременни операции.

3.1.3. Извличане на линковете и нормализирането им

Извличане на линковете и нормализирането им се извършва основно във функцията `extractLinks` в `worker/source/links.ts` (Фиг. 3.1.3.1).

```
const $ = cheerio.load(html);
const seen = new Set<string>();
const normalizedCurrentUrl = normalizeCurrentUrl(pageUrl);
const contentSelector = getContentSelector($);
let linkElements = markSkipSectionsAndGetLinkElements($, contentSelector);
if (linkElements.length === 0) {
  linkElements = contentSelector.find('a[href]').length > 0 ? contentSelector.find('a[href]') : $('a[href]');
}
const links: string[] = [];
```

Фиг. 3.1.3.1

HTML съдържанието се зарежда чрез Cheerio, което позволява DOM-подобна обработка на страницата от страна на сървъра. Избират се всички `<a>` елементи в основната част на съдържанието (с fallback към целия документ при нужда), като се използва Set структура за предотвратяване на дублиране на линкове още на етап извличане.

Функцията `markSkipSectionsAndGetLinkElements` изключва линкове, намиращи се под заглавия като „References“, „Citations“ и сходни секции, често срещани в академични и енциклопедични сайтове. Тези части обикновено съдържат вторични или слабо релевантни препратки, които биха разширили графа с ниска информационна стойност и биха намалили ефективността на обхождането.

```
linkElements.each( (_, element) => {  
  const href = $(element).attr('href');  
  if (!href) return;  
  
  const parsed = normalizeLinkUrl(href.trim(), pageUrl);  
  if (!parsed) return;  
  const { url: linkUrl, normalized: normalizedUrl } = parsed;  
  
  if (seen.has(normalizedUrl)) return;  
  seen.add(normalizedUrl);  
  if (shouldSkipLinkUrl(linkUrl, normalizedCurrentUrl, source)) return;  
  
  links.push(normalizedUrl);  
});
```

Фиг. 3.1.3.2

За всеки открит линк се извършва последователна обработка: нормализиране на URL адреса, проверка за вече срещани стойности и прилагане на допълнителни филтриращи правила. Функцията `normalizeLinkUrl` премахва hash-фрагменти, query параметри и излишни наклонени черти от URL адресите, а `shouldSkipLinkUrl` филтрира нежелани връзки – като self-линкове, външни домейни (при активирано ограничение), PDF файлове и неподдържани протоколи (Фиг. 3.1.3.2). Само връзките, които преминат всички проверки, се добавят към крайния списък, който служи като вход за следващия етап от обхождането.

3.1.4. Crawling

```
export async function crawlPage(  
  url: string,  
  source: Source,  
  conversationId: string  
) : Promise<{ page: Page; html: string } | null> {
```

```
  const response = await fetch(url, {  
    headers: {  
      'User-Agent': 'ScholiaCrawler/1.0',  
    },  
  });
```

Фиг. 3.1.4.1

Страницата се извлича чрез HTTP заявка с изрично зададен User-Agent (Фиг. 3.1.4.1), което позволява коректна идентификация на crawler-а и спазване на добрите практики при автоматизирано обхождане на уеб съдържание.

```
export const MAIN_CONTENT_SELECTOR = 'main, article, .content, #content, #bodyContent, .mw-parser-output';  
  
const mainContent = $(MAIN_CONTENT_SELECTOR).first();  
const mainText = (mainContent.length > 0 ? mainContent.text() : $('body').text()).trim().substring(0, MAX_PAGE_CONTENT_LENGTH);  
const content = mainText || $('body').text().trim().substring(0, MAX_PAGE_CONTENT_LENGTH);
```

Фиг. 3.1.4.2

Тази част от кода гарантира, че в повечето случаи за индексирание се използва възможно най-релевантната текстова част от страницата (спрямо типични семантични CSS селектори, използващи се в сайтове), като същевременно се осигурява надежден fallback механизъм при нестандартна HTML структура (на цялото съдържание на „body“ компонента) (Фиг. 3.1.4.2). Ограничението до 50 000 символа контролира размера на данните и предотвратява прекомерно натоварване при последващата обработка и embedding.

```
const rawTitle = $('title').first().text().trim() ||  
  $('h1').first().text().trim() ||  
  'Untitled';  
const title = rawTitle.replace(PAGE_TITLE_SUFFIX_REGEX, '').trim() || rawTitle;
```

Фиг. 3.1.4.3

Чрез PAGE_TITLE_SUFFIX_REGEX константата се премахват често срещани суфикси изградени от заглавията на страниците (напр. „- Wikipedia“, „-

BBC“, „| Medium“ и др.) (Фиг. 3.1.4.3). Regex изразът е съобразен с популярни сайтове, върху които приложението най-често би се използвало (като Wikipedia, NHS, BBC, Medium и др.), с цел да се получат по-кратки и семантично чисти заглавия. Това подобрява четимостта и визуалната яснота на графа.

3.1.5. Recrawling

```
export async function recrawlSource(conversationId: string, sourceId: string): Promise<void> {
```

Фиг. 3.1.5.1

Съществува и `recrawl` функционалност, която позволява пълно нулиране на съществуващите данни за даден източник (страници, ребра, `chunk`-ове и свързани записи) и създаване на нова задача за обхождане, когато е необходимо повторно индексирание или инспектиране на процеса (Фиг. 3.1.5.1).

3.2. RAG Indexing

```
try {  
  await indexSourceForRag(source.id, job.id, conversationId);  
} catch (err) {  
  console.warn('crawl: RAG indexing failed', err);  
}
```

Фиг. 3.2.1

След като бъде извлечено съдържанието и бъдат установени връзките към нови страници, е необходимо тази информация да бъде оптимизирана за последващи RAG заявки (Фиг. 3.2.1). Директното подаване на целия корпус към езиковия модел би довело до изчерпване на контекстния прозорец и до значително увеличаване на разходите при всяка заявка, което го прави непрактично.

Поради това се използва стратегията Retrieval-Augmented Generation (RAG), при която моделът работи като селектира релевантни текстови сегменти. По-долу е описан детайлно процесът на реализиране на тази стратегия в системата.

3.2.1. Chunking

```
const textSplitter = new RecursiveCharacterTextSplitter({  
  chunkSize: CHUNK_MAX_CHARS,  
  chunkOverlap: CHUNK_OVERLAP_CHARS,  
});
```

Фиг. 3.2.1.1



Целта при chunk-ването е текстът да бъде разделен на семантично „усвоими“ сегменти, като всеки chunk обхваща относително завършена мисъл. Ако в един chunk бъдат обединени две несвързани теми, това може да влоши качеството на embedding-а и съответно да намали ефективността на RAG механизма. От друга страна, прекалено малките сегменти водят до нестабилни или бедни представяния, тъй като липсва достатъчно контекст. Най-добри резултати се постигат, когато chunk-овете улавят завършена идея – без да се разделят изречения или параграфи. Поради тази причина подходящо решение е използването на адаптивен алгоритъм за разделяне, като например Recursive Character Text Splitter от LangChain, който се стреми да запази естествената структура на текста при спазване на зададени ограничения за дължина (Фиг. 3.2.1.1).

```
async function buildChunkSpecsFromPages(  
  pages: { id: string; content: string | null; owner_id: string }[]  
) : Promise<ChunkSpec[]> {  
  const chunkSpecs: ChunkSpec[] = [];  
  for (const page of pages) {  
    const text = (page.content || '').trim();  
    if (!text) continue;  
    const pageChunks = await textSplitter.splitText(text);  
    for (const content of pageChunks) {  
      chunkSpecs.push({  
        page_id: page.id,  
        content,  
        start_index: null,  
        end_index: null,  
        owner_id: page.owner_id,  
      });  
    }  
  }  
  return chunkSpecs;  
}
```

Фиг. 3.2.1.2

Конфигурираме text splitter-а така, че максималната дължина на един chunk да бъде 600 символа, а припокриването между съседни chunk-ове да бъде 100 символа. Това припокриване осигурява плавен преход на контекста между сегментите и намалява риска от загуба на смисъл в границите между тях (Фиг. 3.2.1.2).

3.2.2. Batch embedding

```

async function embedAndInsertChunks(
  chunkSpecs: ChunkSpec[],
  apiKey: string,
  options: { crawlJobId?: string; onProgress?: (inserted: number) => Promise<void> }
): Promise<number> {
  let inserted = 0;
  for (let i = 0; i < chunkSpecs.length; i += EMBED_BATCH_SIZE) {
    const batchSpecs = chunkSpecs.slice(i, i + EMBED_BATCH_SIZE);
    const texts = batchSpecs.map((c) => c.content);
    const embeddings = await embedBatch(apiKey, texts);
    inserted += embeddings.length;
    if (options.onProgress) {
      await options.onProgress(inserted);
    }
  }
  return inserted;
}

async function embedBatch(apiKey: string, texts: string[]): Promise<number[][]> {
  const out: number[][] = [];
  for (let i = 0; i < texts.length; i += EMBED_BATCH_SIZE) {
    const batch = texts.slice(i, i + EMBED_BATCH_SIZE);
    const res = await fetch(OPENAI_EMBEDDINGS_URL, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${apiKey}`,
      },
      body: JSON.stringify({
        model: OPENAI_EMBEDDING_MODEL,
        input: batch,
      }),
    });
    const data = await res.json();
    for (const item of data.data) {
      out.push(item.embedding);
    }
  }
  return out;
}

```

Фиг. 3.2.2.1

За да се избегне претоварване на модела и да се оптимизират външните заявки, chunk-овете се изпращат към embedding модела text-embedding-3-small на OpenAI на партии от по 10 (Фиг. 3.2.2.1).

```

const { error } = await supabase.from('chunks').insert(rows);

```

Фиг. 3.2.2.2

Всеки chunk се преобразува в отделен векторен embedding, който се съхранява в таблицата chunks и впоследствие се използва при RAG заявките като част от общата семантична база знания за съответния разговор (Фиг. 3.2.2.2).

3.3. Извличане на релевантни текстови сегменти чрез RAG и разпределение на chunk-овете за всяка заявка

```
export async function doRetrieve(  
  supabase: SupabaseClient,  
  openaiKey: string,  
  pageIds: string[],  
  queries: string[],  
  perQuery = MATCH_CHUNKS_PER_QUERY,  
): Promise<RetrieveResult> {  
  const embeddings = await embedBatch(openaiKey, queries);
```

Фиг. 3.3.1

Всяка подзаявка (получена след предварителна обработка и декомпозиция на първоначалния потребителски въпрос) се преобразува във векторен embedding чрез същия модел, използван при индексиранието на съдържанието (Фиг. 3.3.1).

```
const { data: matchedChunks } = await supabase.rpc('match_chunks', {  
  query_embedding: embeddings[i],  
  match_page_ids: pageIds,  
  match_count: perQuery,  
});
```

Фиг. 3.3.2

Embedding-ът на подзаявката се съпоставя с embedding-ите на текстовите сегменти (chunks), асоциирани с конкретния разговор, чрез бекенд функцията `match_chunks` (Фиг. 3.3.2), извиквана посредством RPC (Remote Procedure Call). Тази функция реализира векторно similarity търсене в базата данни и връща най-близките сегменти (топ 5 или топ 10 спрямо потребителските настройки) въз основа на семантичната им близост до подадената заявка.



```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE INDEX IF NOT EXISTS idx_chunks_embedding_hnsw
ON chunks USING hnsw (embedding vector_cosine_ops)
WHERE embedding IS NOT NULL;

CREATE OR REPLACE FUNCTION match_chunks(
    query_embedding vector(1536),
    match_page_ids uuid[],
    match_count int DEFAULT 10
)
RETURNS TABLE (
    id uuid,
    page_id uuid,
    content text,
    source_id uuid,
    page_title text,
    page_path text,
    page_url text,
    source_domain text,
    distance float
)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.id,
        c.page_id,
        c.content,
        p.source_id,
        p.title AS page_title,
        p.path AS page_path,
        p.url AS page_url,
        s.domain AS source_domain,
        (c.embedding <=> query_embedding) AS distance
    FROM chunks c
    JOIN pages p ON p.id = c.page_id
    JOIN sources s ON s.id = p.source_id
    WHERE c.embedding IS NOT NULL
        AND c.page_id = ANY(match_page_ids)
    ORDER BY c.embedding <=> query_embedding
    LIMIT match_count;
END;
$$;
```

Фиг. 3.3.3

Функцията `match_chunks` е реализирана чрез разширението `pgvector`, което е активирано в базата данни (както е показано в първия кодов фрагмент). Тя използва векторни операции за изчисляване на сходството между `embedding`-а на заявката и `embedding`-ите на съхранените `chunks`.

Конкретно се изчислява `cosine distance` между двата вектора чрез операторите на `pgvector` (`< = >`), което позволява ефективно сравнение на

семантичната близост (Фиг. 3.3.3). Колкото по-малка е дистанцията, толкова по-релевантен се счита съответният текстов сегмент спрямо подзаявката.

```
CREATE INDEX IF NOT EXISTS idx_chunks_embedding_hnsw
ON chunks USING hnsw (embedding vector_cosine_ops)
WHERE embedding IS NOT NULL;
```

Фиг. 3.3.4

За да не се изчислява сходство между заявката и всеки отделен вектор поотделно, което би било твърде бавно при големи обеми данни, е използван HNSW (Hierarchical Navigable Small World) индекс (Фиг. 3.3.4) [3.3–1]. Това е графово базирана структура за търсене на най-близки съседи, която позволява бързо приближено намиране на най-релевантните вектори, без да се налага пълно сравнение с всички записи в базата.

```
const list = (matchedChunks || []) as ChunkRow[];
chunksPerSubquery.push(list.length);
chunksByQueryIndex.push(list);
for (const c of list) {
  const dist = distanceOf(c);
  const existing = chunkMap.get(c.id);
  if (!existing || distanceOf(existing) > dist) {
    chunkMap.set(c.id, { ...c, distance: dist });
  }
}
```

Фиг. 3.3.5

Ако даден chunk бъде върнат от повече от една подзаявка, се запазва само най-доброто му съвпадение (най-малката дистанция), което предотвратява дублиране и изкривяване на резултатите (Фиг. 3.3.5).

```
const chunks = capWithFairAllocation(
  chunkMap,
  chunksByQueryIndex,
  MATCH_CHUNKS_MERGED_CAP,
);
return { chunks, chunksPerSubquery };
```

Фиг. 3.3.6

За да не претоварваме езиковия модел с прекомерно количество контекст, е въведен глобален лимит от 45 chunk-а, които могат да бъдат подадени като доказателствен материал (evidence). Вместо да се използват само резултатите от



първите няколко подзаявки и да се игнорират останалите, се прилага механизъм за справедливо разпределение, който балансира избора и определя кои точно 45 chunk-а да бъдат включени, така че всяка подзаявка да бъде адекватно представена в крайния контекст (Фиг. 3.3.6).

```
export function capWithFairAllocation<T>(  
  map: Map<string, T>,  
  groups: T[][],  
  cap: number,  
  getKey: (t: T) => string,  
  getDistance: (t: T) => number,  
) : T[] {  
  const numQueries = groups.length;  
  if (numQueries === 0) return [];  
  const perQueryQuota = Math.max(1, Math.floor(cap / numQueries));  
  const selectedKeys = new Set<string>();  
  
  for (let i = 0; i < groups.length; i++) {  
    const list = groups[i].slice().sort((a, b) => getDistance(a) - getDistance(b));  
    let taken = 0;  
    for (const item of list) {  
      if (taken >= perQueryQuota) break;  
      const key = getKey(item);  
      if (selectedKeys.has(key)) continue;  
      selectedKeys.add(key);  
      taken++;  
    }  
  }  
  
  const selected = Array.from(selectedKeys)  
    .map((key) => map.get(key)!)  
    .filter(Boolean)  
    .sort((a, b) => getDistance(a) - getDistance(b));  
  
  if (selected.length >= cap) return selected.slice(0, cap);  
  const remaining = Array.from(map.values())  
    .filter((item) => !selectedKeys.has(getKey(item)))  
    .sort((a, b) => getDistance(a) - getDistance(b));  
  const fill = remaining.slice(0, cap - selected.length);  
  return [...selected, ...fill].sort((a, b) => getDistance(a) - getDistance(b)).slice(0, cap);  
}
```

Фиг. 3.3.7

Функцията `capWithAllocation` разпределя ограничен брой откъси така, че всяка подзаявка да бъде представена с приблизително равен брой най-релевантни резултати (Фиг. 3.3.7). Ако след това останат свободни места под зададения лимит, те се запълват с най-близките по сходство откъси измежду всички



останали (тоест най-правилните отговори спрямо query-тата). Така се избягва ситуация, в която една подзаявка заема почти целия контекст, като същевременно се запазва общата релевантност на избраните резултати.

3.4. Динамичен режим

При създаване на source, потребителят може да избере да го crawl-не по 6 начина. В системата са предвидени четири статични режима на обхождане: Singular (1 страница), Shallow (5 страници), Medium (15 страници) и Deep (35 страници). Те извършват еднократно обхождане с фиксиран лимит и са подходящи за сайтове с относително ограничен брой връзки, като корпоративни страници, продуктови сайтове или техническа документация. При тях процесът е предвидим и ресурсно по-икономичен, тъй като се обхожда ограничен брой страници без допълнителна логика за навигация.

При сайтове с богата вътрешна свързаност, например енциклопедични или информационни платформи, статичният подход често е недостатъчен. Ако се приложи фиксиран лимит (например 35 страници), системата ще обходи началната страница и първите открити връзки, но голяма част от потенциално релевантните страници ще останат необработени. За този тип сценарии е реализиран динамичен режим, при който системата започва от един изходен източник и впоследствие насочва обхождането според конкретните въпроси. Ако наличната информация не е достатъчна за отговор, се предлага следваща страница измежду вече откритите съседни връзки, която с най-голяма вероятност съдържа релевантни данни.

Динамичният режим има два варианта: dynamic surface и dynamic dive. За да се разбере разликата между двата динамични варианта, е важно да се разгледа начинът им на прилагане. Вместо при всяка стъпка всички открити страници да се подават директно към модела с надеждата той сам да избере правилната, което би изразходвало значителни ресурси, кредити и контекстно пространство и би намалило точността, отново се използва RAG механизъм за предварителна селекция. Чрез него се извличат общо десет най-семантично релевантни предложения (спрямо зададените въпроси), които се подават към модела за допълнителна преценка и избор на следваща страница за обхождане, ако е нужно.

При dynamic surface режима се използва кратък текстов откъс от обкръжението на връзката в текущата страница, който се индексира и служи за

оценка на релевантността. При dive режима се извлича и анализира начална част от съдържанието на самата открита страница. Вторият подход изисква допълнително обхождане и обработка, което го прави по-бавен и по-ресурсоемък, но обикновено води до по-точни предложения.

Поради тази разлика потребителят има възможност за избор. По-бързият surface режим е по-подходящ за големи сайтове с множество връзки, където е важно бързо ориентиране в структурата. По-бавният dive режим е по-удачен за по-малки сайтове или в случаи, когато се търси по-прецизен анализ и потребителят може да си позволи по-дълго изчакване. Така системата балансира между скорост и точност според конкретния сценарий на използване.

3.4.1. Encoded Discovered – Dynamic Surface и Dive Имплементация

„Encoded Discovered“ е третият показател в статистиката над граф визуализацията, наред с „Indexed“ и „Discovered“. Той отразява в реално време броя на откритите връзки, които са преминали през семантично кодиране и могат да бъдат използвани като предложения за разширяване на корпуса. Индикаторът се визуализира само когато към разговора е добавен източник в динамичен режим, тъй като именно тогава се генерират и обработват такива кандидат-страници.

3.4.1.1. Dynamic Surface

```
const isDynamic = source.crawl_depth === 'dynamic';  
const isSurface = (source as { suggestion_mode?: string }).suggestion_mode !== 'dive';  
const links = extractLinks(html, normalizedUrl, source);  
const linksWithContext = isDynamic && isSurface ? extractLinksWithContext(html, normalizedUrl, source) : [];
```

Фиг. 3.4.1.1.1

В динамичния surface режим, за разлика от статичния, не се извличат единствено свързаните линкове, а и текстовия контекст около тях чрез функцията `extractLinksWithContext` (Фиг. 3.4.1.1.2). Това позволява всяка открита връзка да бъде оценявана не само по самото си наличие, а и спрямо смисловата среда, в която се появява.

```
export function extractLinksWithContext(
```

```
const encodedToInsert = toEncode
  .filter((l) => urlToEdgeId.has(l.url))
  .map((l) => ({
    page_edge_id: urlToEdgeId.get(l.url)!,
    anchor_text: l.anchorText || null,
    snippet: l.snippet.substring(0, 500),
    owner_id: source.owner_id,
  }));
if (encodedToInsert.length > 0) {
  const { error: encError } = await supabase.from('encoded_discovered').upsert(encodedToInsert, {
    onConflict: 'page_edge_id',
    ignoreDuplicates: true,
  });
  if (encError) {
    console.warn('crawl: encoded_discovered insert failed', encError.message);
  }
}
```

Фиг. 3.4.1.1.2

Извлеченият контекст се записва в таблицата `encoded_discovered`, която съхранява текстовите откъси, свързани с конкретно ребро (`page_edge`). По този начин всяка връзка в графа разполага със собствен семантичен контекст, който може да бъде използван при последващо търсене и селекция.

3.4.1.2. Dynamic Dive

```
if (useDive) {
  const url = edgeIdToUrl.get(item.page_edge_id) || '';
  const lead = await fetchTargetPageLead(url);
  if (lead) {
    text = lead;
    await supabase
      .from('encoded_discovered')
      .update({ snippet: text })
      .eq('id', item.id);
  }
}
```

Фиг. 3.4.1.2.1

Ако режимът е зададен като `dive`, вместо контекст от текущата страница в таблицата `encoded_discovered` се записва начален откъс (`lead snippet`) от самата открита страница, който служи като по-представителен семантичен индикатор за нейното съдържание (Фиг. 3.4.1.2.1).

3.4.2. Suggestions

```
const topSuggestedPages: SuggestedPage[] | null =
  dynamicMode && sourceIds.length > 0
    ? await getTopSuggestedPages(supabase, openaiKey, sourceIds, userMsg, subqueriesToRun.slice(0, 3), 5)
    : null;
log('extract-call', { iteration, chunkCount: evidenceChunksForExtract.length, topSuggestedCount: topSuggestedPages?.length ?? 0 });
const snippetPreviews = evidenceChunksForExtract.map((q) => (q.snippet ?? '').slice(0, 120));
log('extract-evidence-preview', { iteration, snippetPreviews });
const extractResult = await callExtractAndDecide(
  openaiKey,
  slotRowsForExtract,
  evidenceChunksForExtract,
  currentSlotStateJson,
  userMsg,
  dynamicMode,
  topSuggestedPages,
);
```

Фиг. 3.4.2.1

В процеса на разсъждение при всяка итерация се извличат десетте най-вероятни кандидат-страници от вече индексирани открити връзки (encoded_discovered) чрез семантично търсене (Фиг. 3.4.2.1).

```
const { data: matches, error: rpcErr } = await supabase.rpc('match_discovered_links', {
  query_embedding: queryEmbs[i],
  match_source_ids: sourceIds,
  match_count: 12,
});
```

Фиг. 3.4.2.2

Използваме RPC функция match_discovered_links (Фиг. 3.4.2.2), имплементирана по подобен начин на match_chunks (описана в точка 3.3), за да намерим най-близките предложения (suggestions).

```
export async function getTopSuggestedPages(
  supabase: SupabaseClient,
  openaiKey: string,
  sourceIds: string[],
  userMsg: string,
  subqueriesToRun: string[],
  limit: number,
) {
  const topN = capWithFairAllocation(
    matchMap,
    matchesByQueryIndex,
    limit,
    (x) => `${x.m.source_id}:${x.m.to_url}`,
    (x) => x.distance,
  );
}
```

Фиг. 3.4.2.3

Тъй като един reasoning step може да включва няколко подзаявки, кандидатите се подбират чрез механизъм за справедливо разпределение: всяка подзаявка получава приблизително равен брой от най-релевантните си резултати, а оставащите позиции се запълват с глобално най-близките съвпадения (с най-

ниска дистанция). За целта отново се използва помощната функция `capWithFairAllocation` (описана в т. 3.3) в `getTopSuggestedPages` (Фиг. 3.4.2.3).

```
"next_action": "retrieve" | "expand_corpus" | "clarify" | "answer",
"suggested_page_index": "optional: when next_action is expand_corpus and a candidate list was provided,
integer 1-5 indicating which candidate page to suggest (1 = first); omit to suggest the first"
Candidate suggested pages (non-indexed links; only suggest if one is clearly relevant and evidence lacks the information):
```

Фиг. 3.4.2.4

Получените предложения се подават към модела заедно с останалата част от всеки промпт, който е част от процеса на разсъждение на модела, като той може да избере действие „`expand_corpus`“, при което конкретната избрана от модела страница се предлага за добавяне и последващо обхождане (повече за процеса на мислене на модела в 3.6) (Фиг. 3.4.2.4).

3.4.3. Добавяне на страница към динамичен source

След като предложението излезе на екрана и потребителят го потвърди, нов `crawl job` се създава за предложения линк. Той се добавя като част от динамичния source, от който произлиза (branch out). И `encoding discovered` процеса се пуска наново.

```
export async function processAddPageJob(job: {
  id: string;
  source_id: string;
  explicit_crawl_urls: string[];
}): Promise<void> {
```

Фиг. 3.4.3.1

Използва се колоната `explicit_crawl_urls` в таблицата `crawl_jobs`, която позволява изрично задаване на конкретни URL адреси за обхождане и по този начин отменя стандартните настройки за начална страница и дълбочина на обхождане (Фиг. 3.4.3.1).

3.4.4. Recrawl на динамичен source

```
const explicitKey = (job as { explicit_crawl_urls?: string[] | null }).explicit_crawl_urls;
const seedUrls: string[] =
  explicitKey && explicitKey.length > 0
    ? explicitKey.map((u) => normalizeUrlForCrawl(u))
    : [normalizeUrlForCrawl(source.initial_url)];
```

Фиг. 3.4.4.1



При повторно обхождане (recrawl) на динамичен източник колоната `explicit_crawl_urls` отново се използва за изрично задаване на URL адресите, които трябва да бъдат обходени, като по този начин се гарантира контролирано и пълно повторно индексирание с всички добавени след инициализиране предложени страници (Фиг. 3.4.4.1).

3.5. Supabase Realtime

```
export const useChatDatabase = () => {  
  useRealtimeCrawlUpdates(activeConversationId, sourceIds);
```

Фиг. 3.5.1

Платформата използва Supabase Realtime за абонамент към промени в базата данни, когато потребителят активно разглежда даден разговор. Realtime механизмът се използва единствено за live актуализации, свързани с процеса на обхождане и визуализацията на графа (Фиг. 3.5.1).

Чрез него се следят в реално време статусите на `crawl_jobs` (например `queued`, `running`, `completed`), добавянето на нови страници, създаването на `page_edges`, както и записите в `encoded_discovered`, свързани с открити и впоследствие кодирани линкове. По този начин интерфейсет отразява динамично текущия напредък на обхождането и изграждането на `knowledge graph`-а.

За всяка активна сесия се създава отделен канал (`subscription`).

3.5.1. Crawl_jobs канал


```
if (sourceIds.length > 0) {
  const crawlJobsChannel = supabase
    .channel(`crawl-jobs:${conversationId}`)
    .on<CrawlJob>('postgres_changes', {
      event: '*',
      schema: 'public',
      table: 'crawl_jobs',
      filter: `source_id=in.${sourceIds.join(',')}`,
    },
    (payload) => {
      const job = payload.new as CrawlJob;
      const predicate = (query: { queryKey: unknown }) => {
        const key = query.queryKey;
        if (!Array.isArray(key)) return false;
        const [prefix, arg1, arg2] = key;
        if (queryKeyMatches(key, CURRENT_CRAWL_JOB_BY_SOURCE, job.source_id)) return true;
        if (prefix === LIST_OF_CRAWL_JOBS_BY_SOURCE && (arg1 === job.source_id || key.length === 1)) return true;
        if (CRAWL_JOB_INVALIDATION_PREFIXES.includes(prefix as (typeof CRAWL_JOB_INVALIDATION_PREFIXES)[number])) return true;
        if (prefix === LATEST_ADD_PAGE_JOB_BY_CONVERSATION_AND_SOURCE && arg1 === conversationId && arg2 === job.source_id && job.explicit_crawl_urls != null) return true;
        return false;
      };
      queryClient.invalidateQueries({ predicate });
      queryClient.invalidateQueries({ queryKey: [SOURCES_FOR_CONVERSATION, conversationId] });
    })
    .subscribe();
  channelsRef.current.push(crawlJobsChannel);
}
```

Фиг. 3.5.1.1

Създава се отделен realtime канал с име `crawl-jobs:${conversationId}`, което позволява логическо изолиране на събитията по разговор (Фиг. 3.5.1.1). Абонаментът е към `postgres_changes` върху таблицата `crawl_jobs`, като се следят всички събития (`event: '*'`): `insert`, `update` и `delete`. Използва се филтър `source_id=in(...)`, така че клиентът да получава само събития за `crawl jobs`, принадлежащи на `source`-овете в текущия `conversation`. При получаване на събитие се извлича обновения запис (`payload.new`) и се създава `predicate` функция за селективна инвалидизация на React Query кеша. Вместо да се инвалидира целия кеш, логиката проверява кои `query` ключове са засегнати от промяната и инвалидира само тях (Фиг. 3.5.1.2).

```
export const CURRENT_CRAWL_JOB_BY_SOURCE = 'current-crawl-job-by-source';
export const LIST_OF_CRAWL_JOBS_BY_SOURCE = 'list-of-crawl-jobs-by-source';
export const LATEST_MAIN_CRAWL_JOB_BY_SOURCES = 'latest-main-crawl-job-by-sources';
export const CRAWL_JOB_INVALIDATION_PREFIXES = [
  CURRENT_CRAWL_JOB_BY_SOURCE,
  LIST_OF_CRAWL_JOBS_BY_SOURCE,
  LATEST_MAIN_CRAWL_JOB_BY_SOURCES,
] as const;
```

Фиг. 3.5.1.2

3.5.2. Page_edges канал – Граф

```
const edgesChannel = supabase
  .channel(`page-edges:${conversationId}`)
  .on<PageEdge>('postgres_changes',
    {
      event: 'INSERT',
      schema: 'public',
      table: 'page_edges',
    },
    () => {
      const edgesPredicate = (query: { queryKey: unknown }) =>
        queryKeyMatches(query.queryKey, PAGE_EDGES_FOR_CONVERSATION, conversationId);
      const doSync = () => {
        const t = Date.now();
        if (import.meta.env.DEV) {
          const since = edgesLastSyncRef.current ? t - edgesLastSyncRef.current : 0;
          console.log('[realtime] edges sync', since ? `${since}ms since last` : 'initial');
        }
        edgesLastSyncRef.current = t;
        queryClient.invalidateQueries({ predicate: edgesPredicate });
        queryClient.refetchQueries({ predicate: edgesPredicate });
      };
      const now = Date.now();
      if (edgesLastSyncRef.current === 0 || now - edgesLastSyncRef.current >= EDGES_DEBOUNCE_MS) {
        doSync();
      }
      if (edgesDebounceTimerRef.current) clearTimeout(edgesDebounceTimerRef.current);
      edgesDebounceTimerRef.current = setTimeout(() => {
        edgesDebounceTimerRef.current = null;
        doSync();
      }, EDGES_TRAILING_MS);
    }
  )
  .subscribe((status) => {
    if (status === 'SUBSCRIBED') syncAfterSubscribe();
  });
channelsRef.current.push(edgesChannel);
```

Фиг. 3.5.2.1

Частта, отговаряща за page_edges, реализира абонамент към всички нови връзки между страници, които се създават по време на обхождане (Фиг. 3.5.2.1). Тези връзки моделират графовата структура на индексираното съдържание.

Създава се realtime канал с име page-edges:\${conversationId}, който се абонира за INSERT събития в таблицата page_edges. Тук не се използва филтър по source_id или conversation_id. Причината е, че релацията към conversation вече се управлява на ниво заявка чрез RLS политики в базата, а при всяко ново edge събитие така или иначе се извършва селективна инвалидизация на конкретните заявки, свързани с текущия разговор.



По време на crawl процеса може да се генерират хиляди `page_edges` записи за кратък период (например при дълбочинно обхождане). Това води до потенциален „event flood“ от realtime събития. За да се предотврати прекомерно инвалидизиране на кеша и претоварване на браузъра, е имплементиран механизъм за `debounce` (Фиг. 3.5.2.2).

При всяко `INSERT` събитие се дефинира `predicate`, който селектира само заявките с ключ `PAGE_EDGES_FOR_CONVERSATION` за текущия `conversationId`. Вместо да се извиква незабавно `invalidateQueries` за всяко събитие, се използва двустепенна стратегия:

Първо, ако от последната синхронизация са изминали повече от `EDGES_DEBOUNCE_MS` (1.2 секунди), се изпълнява незабавна синхронизация чрез функцията `doSync`. Тя инвалидизира и след това извлича повторно съответните заявки, така че графът да бъде актуализиран.

Второ, независимо от това, се стартира „trailing“ таймер (`EDGES_TRAILING_MS` = 800 ms). Ако в този интервал не пристигнат нови събития, се изпълнява синхронизация. По този начин се гарантира, че след приключване на `burst` от събития интерфейсът ще отрази окончателното състояние.

В резултат се постига баланс между реактивност и стабилност. Интерфейсът се обновява в почти реално време, но без да се създава прекомерно натоварване върху клиента или мрежата при масово създаване на `edges` по време на активно обхождане.

```
const EDGES_DEBOUNCE_MS = 1200;  
const EDGES_TRAILING_MS = 800;  
const DISCOVERED_LINKS_DEBOUNCE_MS = 500;  
const PAGES_DEBOUNCE_MS = 300;
```

Фиг. 3.5.2.2

Същият механизъм се прилага и при канала за `encoded_discovered`, както и при `pages INSERT` събитията. При откриването на всяка нова `discovered` страница (в динамичен режим), които изчисляват броя на откритите и вече кодирани линкове. По този начин прогрес барът отразява в реално време различните етапи на обработка – `Scraping`, `Indexing` и `Encoding Discovered`, като последният етап е приложим единствено при динамичен режим на обхождане.

Аналогично, при pages INSERT събитията се инвалидизират заявките, които зареждат страниците за даден source или conversation, което гарантира, че новите възли в графа се визуализират незабавно. Това осигурява синхрон между backend процеса на обхождане и frontend представянето на текущото състояние.

3.6. Главен Алгоритъм – Алгоритъм за многостепенно разсъждение

Това е най-оригиналната и обемна част от системата Scholia. Функцията chat-with-rag (разположена в директорията ./supabase/functions/chat-with-rag) представлява Supabase Edge Function, който управлява цялостната логика на модела – от обработката на входната потребителска заявка, през планирането и итеративното извличане на доказателства, до финализирането на отговора с проследими цитати. Именно тук се реализира механизмът за структурирано разсъждение, контрол на completeness, управление на подзаявки, динамично разширяване на корпуса и синхронизация със слоя за съхранение на reasoning стъпки и доказателства.

3.6.1. Фаза на планиране

Планиращата фаза представлява първата стъпка от изпълнението на модела. При подаване на потребителска заявка тя се изпраща към езиковия модел чрез OpenAI API, преди да бъде извършено каквото и да е извличане или обработка на данни (Фиг. 3.6.1.1). Целта на този етап е да се генерира структурирана стратегия за по-нататъшното разсъждение.

```
export async function callPlan(apiKey: string, userMessage: string): Promise<PlanResult> {
  const res = await fetch('https://api.openai.com/v1/chat/completions', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${apiKey}` },
    body: JSON.stringify({
      model: OPENAI_CHAT_MODEL,
      messages: [
        { role: 'system', content: PLAN_SYSTEM },
        { role: 'user', content: userMessage },
      ],
      response_format: { type: 'json_object' },
    }),
  });
  // ...
  - slots: array of { name, type, description?, required?, dependsOn?, target_item_count?, items_per_key? }
```

Фиг. 3.6.1.1



Основната цел на планиращата фаза е да идентифицира така наречените „слотове“. Те представляват структурирани placeholders за информация, които ще бъдат постепенно попълвани чрез RAG заявки към индексирания корпус от страници. Всеки слот моделира логически елемент от финалния отговор и може да бъде подкрепян с доказателства (evidence quotes), извлечени от базата. Вместо системата да генерира директен текстов отговор, тя първо разлага въпроса на отделни информационни компоненти, които се обработват поетапно и проследимо.

```
const PLAN_SYSTEM = `You plan semantic search and evidence gathering for a question over indexed documents.

Output JSON only with this shape:
- action: "retrieve" | "clarify"
- why: short reason for this action
- slots: array of { name, type, description?, required?, dependsOn?, target_item_count?, items_per_key? }
  - type is one of: "scalar" (one value), "list" (set of items), "mapping" (key->value per list item; use dependsOn: slot name of the list)

  - dependsOn: slot whose extracted values are required to build this slot's query.
  Use only when independent querying can't be meaningfully done without those values. (mapping always depends on a list)

  - description: one short sentence for what this slot represents (helps extraction and UI)

  - target_item_count: for list slots only. Set to the number of items the user asked for (e.g. "top 5 products" -> 5).
  Set to 0 if the user did not specify a concrete number. Omit or 0 for scalar/mapping.

  - items_per_key: for mapping slots only. Number of values per key (e.g. "top 2 achievements per product" -> 2).
  Backend computes total target from dependency list target_item_count x items_per_key.

- subqueries: array of { slot, query } – only for slots that have no dependencies (omit dependsOn).
Each query is a search phrase for the slot. Do not include subqueries for mapping slots or any slot that dependsOn another; those are run later once dependencies are filled.

Rules:
- Start with action "retrieve" unless the question is ambiguous (then "clarify" with questions).
- Subqueries: only for slots with no dependencies (scalars and lists that do not dependOn another slot).
For scalar slots use 1-2 focused queries. For list slots use 1-2 high-level discovery (BROAD) queries (e.g. "company product list", "Biden major achievements").`;
```

Фиг. 3.6.1.2

Слововете могат да бъдат попълвани итеративно. В зависимост от резултатите от предходните стъпки моделът може да преформулира заявки, да използва различни стратегии на търсене или да изгради „верига“ от зависимости между отделните информационни единици (Фиг. 3.6.1.2). Това позволява постепенно акумулиране на знание, вместо еднократно и неконтролируемо извличане.

```
- type is one of: "scalar" (one value), "list" (set of items), "mapping" (key->value per list item; use dependsOn: slot name of the list)
```

Фиг. 3.6.1.3

Системата поддържа три типа слотове (Фиг. 3.6.1.3). Първият тип е скаларен слот. Той съдържа единична атомарна информационна единица – например „кой“, „кога“, „къде“, „какво“. Типични примери са „Кой е министър-председател на България през 2001 г.“ или „Кога е основан Софийският университет?“. В този случай слотът съдържа само един slot_item.



Вторият тип е списъчен (list) слот. Той представлява колекция от скаларни стойности и позволява акумулиране на множество резултати през различни RAG итерации. Например въпрос като „Кои са най-значителните постижения на Барак Обама?“ предполага множество отделни информационни единици, които могат да бъдат извлечени постепенно чрез различни подзаявки. Всеки елемент от списъка се съхранява като отделен `slot_item`.

```
- target_item_count: for list slots only. Set to the number of items the user asked for (e.g. "top 5 products" -> 5).  
Set to 0 if the user did not specify a concrete number. Omit or 0 for scalar/mapping.
```

Фиг. 3.6.1.4

List слотовете могат да имат и параметър `target_item_count` (Фиг. 3.6.1.4), когато потребителят изрично е посочил конкретен брой елементи (например „пет причини“, „три примера“ и т.н.). Този параметър служи като ориентир за completeness и указва на модела кога списъкът може да се счита за достатъчно попълнен.

Третият тип е mapping слот. Той се използва при по-комплексни въпроси от типа „за всеки X какво е Y“. Тук данните се съхраняват като key-value двойки. Например при въпрос „За всеки министър-председател на България след 1990 г. коя е неговата партия?“ ключовете са имената на министрите, а стойностите – съответните партии. Mapping слотовете позволяват структурирано представяне на съпоставима информация.

```
- items_per_key: for mapping slots only. Number of values per key (e.g. "top 2 achievements per product" -> 2).  
Backend computes total target from dependency list target_item_count x items_per_key.
```

Фиг. 3.6.1.5

Параметърът `items_per_key`, заедно с `target_item_count` на списъчния слот, от който mapping слотът зависи, се използват при оценката на completeness (Фиг. 3.6.1.5). Те определят колко стойности трябва да бъдат намерени за всеки ключ и общо за всички ключове, така че mapping слотът да се счита за достатъчно попълнен.

Попълването на слотовете се осъществява чрез `slot_item` записи. При скаларните слотове съществува само един `slot_item`. При list слотовете се поддържа масив от скаларни `slot_item`-и. При mapping слотовете `slot_item`-ите



представяват key–value двойки. Всеки slot_item трябва да бъде подкрепен с поне един цитат от базата, което гарантира доказуемост на всяка извлечена стойност.

Слотовете могат да имат зависимости помежду си. Това е ключов механизъм, който позволява каскадно разсъждение. Например при въпрос „Според подадения журнал коя е Мис България 1998, кои са нейните пет сестри и кога е родена всяка?“ първият слот е скаларен – „Мис България 1998“. След като този слот бъде попълнен (например с „Наталия Гуркова“), може да се премине към втория слот – „сестрите на Мис България 1998“, който е от тип list. Този втори слот зависи от първия, защото без да знаем конкретното име, заявката „сестрите на Мис България 1998“ вероятно няма да съвпадне с формулировката в реалните източници. След като бъде установено името, заявките могат да се преформулират в „Наталия Гуркова сестра“, което увеличава вероятността за успешно извличане.

Третият слот в този пример е от тип mapping – „година на раждане на сестрите“. Той зависи от list слота със сестрите, тъй като за всяка сестра ще бъде изпълнена отделна заявка от типа „[име на сестра] година на раждане“. По този начин се реализира многостепенна верига на разсъждение: първо се идентифицира основния обект, след това свързаните обекти, а накрая техни конкретни атрибути.

- subqueries: array of { slot, query } – only for slots that have no dependencies (omit dependsOn).
Each query is a search phrase for the slot. Do not include subqueries for mapping slots or any slot that dependsOn another; those are run later once dependencies are filled.

Фиг. 3.6.1.6

В рамките на планиращата фаза езиковият модел генерира и набор от преформулирани подзаявки, оптимизирани за RAG процеса. Потребителската заявка често съдържа сложна или защитна формулировка, която не е подходяща за директно семантично търсене. Затова тя се трансформира в по-кратки, целево насочени заявки, които максимизират релевантността на извлечените текстови сегменти. По този начин планиращата фаза не просто разлага въпроса, а конструира контролируем план за извличане и структуриране на знание (Фиг. 3.6.1.7).

- Subqueries: only for slots with no dependencies (scalars and lists that do not dependOn another slot).
For scalar slots use 1 focused queries. For list slots use 2 high-level discovery (BROAD) queries (e.g. "company product list", "Biden major achievements").;

Фиг. 3.6.1.7



3.6.2. Цикъл на мислене и итерациите му

След приключване на планиращата фаза започва итеративният цикъл на изпълнение. В рамките на този loop системата последователно изпълнява RAG заявки, извлича доказателства и попълва съответните слотове, като след всяка итерация се изчислява степента на completeness. Цикълът продължава, докато всички задължителни слотове бъдат достатъчно попълнени и може да се генерира финален отговор. Ако обаче прогресът по попълването стигне до застой (stagnation) и не се добавя нова релевантна информация, процесът се прекратява и се връща частичен отговор, ясно обозначен като такъв.

```
const { chunks: retrievedChunks, chunksPerSubquery } = await doRetrieve(  
  supabase,  
  openaiKey,  
  pageIds,  
  subqueriesToRun,  
);
```

Фиг. 3.6.2.1

Първата стъпка от итеративния цикъл е фазата retrieve (Фиг. 3.6.2.1). В нея се изпълняват RAG подзаявките, генерирани в планиращата фаза, като върху индексирания корпус се извършва семантично търсене и се извличат най-релевантните текстови сегменти (chunks). Тези резултати представляват доказателствения материал за текущата итерация и се подават към следващата фаза на извличане и структуриране на твърденията (подробно описано в раздел 3.3).

export async function callExtractAndDecide(

```
const EXTRACT_SYSTEM = `You extract atomic claims from the provided evidence (chunks) and decide the next step.

Output JSON only:
{
  "claims": [
    { "slot": "slot_name", "value": <atomic value: string or number>, "key": "<only for mapping slots>", "confidence": 0.0-1.0, "chunkIds": ["chunk-uuid-1", "chunk-uuid-2"] }
  ],
  "next_action": "retrieve" | "expand_corpus" | "clarify" | "answer",
  "why": "short reason",
  "subqueries": "optional: when next_action is retrieve, array of { \"slot\": \"slot_name\", \"query\": \"search phrase\" } for the next retrieval round",
  "questions": "optional: when next_action is clarify, array of clarifying question strings",
  "suggested_page_index": "optional: when next_action is expand_corpus and a candidate list was provided, integer 1-10 (1 = first); omit for first",
  "broad_query_completed_slot_fully": "optional: array of BROAD slot names (listed below) for which no more retrieval is needed; evidence sufficient."
}

Rules:
- Claims: only from given chunks; each claim must list at least one chunkId (exact UUID from [uuid] lines).
- Scalar: one value, no key. List: one claim per item. Mapping: key = one of the dependency slot's entity names from current slot state; do not invent keys.

- Prefer "retrieve" or "answer"; use "expand_corpus" only when evidence genuinely lacks the facts (not merely spread across chunks).
- Use "clarify" only when the question is ambiguous, not when evidence is missing.

- Answer: Set next_action to "answer" when required slots are finished/complete or retrieval has stagnated with partial evidence.
- Backend runs a separate final-answer step; you do not write answer text.

- Subqueries: omit for (a) slots that have finished querying (listed below), (b) scalar slots that already have a value in current slot state,
(c) list/mapping slots that have reached target (see "Slots to fill" targets; target 0 = no fixed target, continue until broad_query_completed_slot_fully or stagnate).
Only suggest subqueries for slots that still need retrieval after your claims.

- BROAD vs TARGETED: Backend lists BROAD slots this step. Use broad-style only for those; targeted for other list/mapping.
For BROAD slots you may set broad_query_completed_slot_fully if no more retrieval needed. Never repeat an identical query; use "last queries and items" to try something different.

- Candidate suggested pages: prefer "expand_corpus" only when evidence genuinely lacks info AND a candidate is clearly relevant;
otherwise "retrieve" with subqueries or "answer". If expand_corpus, set suggested_page_index (1-10) or omit for first.`;
```

Фиг. 3.6.2.2

Втората стъпка е фазата extraction (Фиг. 3.6.2.2). В нея извлечените от RAG текстови сегменти се подават към езиковия модел с цел структурирано извличане на твърдения (claims), които могат да бъдат съпоставени със съществуващите слотове. Моделът идентифицира кои слотове могат да бъдат попълнени въз основа на наличните доказателства и при необходимост предлага нови подзаявки за онези слотове, които все още са непълни.

Получените claims се валидират спрямо типа на съответния слот (скаларен, list или mapping) и се трансформират в записи в таблицата slot_items. По този начин всяка извлечена информационна единица се материализира като структурирана стойност в базата данни, обвързана с конкретни доказателствени цитати.

```
dynamicBlock = `
Candidate suggested pages (suggest expand_corpus only if one is clearly relevant and evidence lacks the info):
${candidateList}

expand_corpus: set suggested_page_index (1-${topSuggestedPages.length}) or omit for first. Prefer retrieve when more retrieval could fill slots.`;
```

Фиг. 3.6.2.3

В тази фаза, ако системата работи в динамичен режим, към модела се подават и възможните предложения за разширяване на корпуса (suggested pages) (Фиг. 3.6.2.3). Това позволява на модела да прецени дали наличните доказателства

са достатъчни или е необходимо да се избере действие от тип expand corpus, като се предложи добавяне на конкретна страница с висока вероятност за релевантност.

3.6.2.1. BROAD срещу TARGETED subquery-та

```
const strategy: 'broad' | 'targeted' | null =
  slot && (slot.type === 'list' || slot.type === 'mapping') ? (slot.attempt_count > 0 ? 'targeted' : 'broad') : null;
```

Фиг. 3.6.2.1.1

BROAD заявките се изпълняват за list и mapping слотовете само по веднъж и едва след като всички техни зависимости са изпълнени (т.е. когато attempt_count за съответния слот все още е нула) (Фиг. 3.6.2.1.1). Това представлява първоначалната фаза за запълване на тези слотове – аналогична на планиращата логика при независимите слотове.

Целта на BROAD заявката е да се провери дали търсената информация съществува в естествено агрегирана форма в корпуса – например формулировки от типа „Продуктите, които компанията предлага, са ..., ... и ...“. Ако подобен обобщаващ фрагмент бъде открит, слотът може да бъде попълнен наведнъж, без да е необходимо изпълнение на множество последващи таргетирани заявки за отделните елементи.

```
"broad_query_completed_slot_fully": "optional: array of BROAD slot names (listed below) for which no more retrieval is needed; evidence sufficient."
```

Фиг. 3.6.2.1.2

Ако моделът прецени, че резултатът от BROAD заявката съдържа директно и изчерпателно формулиран отговор, който съвпада с търсената от потребителя информация, той маркира съответния слот като напълно попълнен чрез задаване на полето broad_query_completed_slot_fully (Фиг. 3.6.2.1.2). Това означава, че не са необходими допълнителни таргетирани заявки за този слот и той се счита за финализиран още в началната фаза.

```
- BROAD vs TARGETED: Backend lists BROAD slots this step. Use broad-style only for those; targeted for other list/mapping.
For BROAD slots you may set broad_query_completed_slot_fully if no more retrieval needed. Never repeat an identical query; use "last queries and items" to try something different.
```

Фиг. 3.6.2.1.3

Ако BROAD заявката не доведе до достатъчно информация – например при въпрос като „Кои са най-значителните постижения на Обама?“ – се преминава към по-таргетирани заявки (Фиг. 3.6.2.1.3). Причината е, че подобни списъци



рядко са формулирани в един единствен обобщаващ абзац. По-вероятно е отделните постижения да бъдат описани в различни части на текста.

В такъв случай системата генерира подзаявки, които хипотетично свързват обекта с възможни тематични направления – например „Obama healthcare“, „Obama foreign policy“, „Obama economic reforms“ и др. По този начин информацията се акумулира постепенно от различни сегменти на корпуса, вместо да се разчита на съществуването на един централен, агрегиращ фрагмент.

3.6.2.2. Кога всеки слот спира да се query-ва

```
Slots to fill:
${slotBlock}

Slots already finished (no subqueries needed):
${finishedBlock}
```

Фиг. 3.6.2.2.1

Във всяка итерация към модела се подават както слотовете, които все още трябва да бъдат попълнени, така и вече финализираните слотове (Фиг. 3.6.2.2.1). Това осигурява контекст за текущото състояние на разсъждениято – както за зависимите слотове, които стъпват върху вече намерена информация, така и като обща картина на вече установените факти. По този начин моделът избягва дублиране на заявки и може да формулира по-прецизни подзаявки, съобразени със съществуващите данни.

```
if ((extractResult.broad_query_completed_slot_fully ?? []).includes(slot.name)) slot.finished_querying = true;
if (currentCount === prevCount) slot.finished_querying = true;
```

Фиг. 3.6.2.2.2

Определянето кои слотове се считат за завършени (finished) се извършва според техния тип и текущото им състояние (Фиг. 3.6.2.2.2).

При скаларните слотове критерият е ясен – ако към съответния слот съществува поне един свързан slot_item, той се счита за попълнен.

При list и mapping слотовете логиката е по-сложна. Те се маркират като finished querying в два основни случая: когато процесът по извличане е достигнал застой (stagnation) и не се добавят нови елементи, или когато при изпълнение на BROAD заявка моделът изрично е оценил, че слотът е напълно попълнен. В последния случай това се отбелязва чрез съответния флаг за завършеност (описан по-подробно в края на раздел 3.6.1).

```
const getEffectiveTarget = (slot: SlotDb, counts: Map<string, number>): number => {
  if (slot.type === 'list') return slot.target_item_count ?? 0;
  if (slot.type === 'mapping' && slot.depends_on_slot_id && slot.items_per_key != null)
    return (counts.get(slot.depends_on_slot_id) ?? 0) * slot.items_per_key;
  return 0;
};

for (const slot of slotsWithAttempts) {
  const count = slotItemCountBySlotId.get(slot.id) ?? 0;
  const effectiveTarget = getEffectiveTarget(slot, slotItemCountBySlotId);
  if (effectiveTarget > 0 && count >= effectiveTarget) slot.finished_querying = true;
}
```

Фиг. 3.6.2.2.3

Допълнително, при list слотовете се проверява дали текущият брой slot_item-и е достигнал зададения target_item_count. Ако потребителят е посочил конкретен брой (например „Намери две постижения на ...“), слотът се маркира като finished querying веднага щом този праг бъде достигнат.

При mapping слотовете оценката е двустепенна. Използва се параметърът items_per_key, който указва колко стойности трябва да бъдат намерени за всеки ключ. Ако mapping слотът зависи от list слот с target_item_count, максималния очакван брой елементи се изчислява като target_item_count × items_per_key. Ако обаче зависимият list слот вече е приключил querying (дори без фиксиран target), тогава се използва текущия му брой елементи, умножен по items_per_key, за да се изчисли очаквания максимум. При достигане на този праг mapping слотът се счита за завършен (Фиг. 3.6.2.2.3).

```
export function overallCompleteness(
  slots: SlotForCompleteness[],
  slotItemCountBySlotId: Map<string, number>,
  slotMetaBySlotId?: Map<string, SlotCompletenessMeta>,
): number {
  const required = slots.filter((s) => s.required);
  if (required.length === 0) return 1;
  let sum = 0;
  let weightSum = 0;
  for (const slot of required) {
    const w = slot.type === 'mapping' ? 2 : 1;
    sum += slotCompleteness(slot, slotItemCountBySlotId, slotMetaBySlotId) * w;
    weightSum += w;
  }
  return weightSum > 0 ? sum / weightSum : 0;
}
```

Фиг. 3.6.2.2.4



За визуализиране на общата completeness в потребителския интерфейс се агрегира степента на попълване на всички слотове спрямо техните целеви изисквания (Фиг. 3.6.2.2.4). По този начин прогресът на целия отговор се представя като функция от това колко от необходимите информационни единици са успешно извлечени и структурирани Decision Logic.

3.6.3. Финален отговор

Моделът прекратява итеративния цикъл и преминава към финализиране на отговора в два основни случая: когато всички задължителни слотове са напълно попълнени и моделът изрично избере действие „answer“, или когато за всеки слот е маркирано finished querying, което означава, че не се очаква допълнително извличане на релевантна информация.

Освен действията retrieve и answer, съществуват още две възможни разклонения в процеса. Първото е expand corpus, което се активира при наличие на релевантни предложения за разширяване на корпуса (вж. 3.4.2) и позволява добавяне на нови страници към източниците с цел извличане на допълнителни доказателства. Второто е clarify, което се задейства, когато моделът установи, че въпросът е неясен или недостатъчно конкретен, и е необходимо допълнително уточнение от страна на потребителя.

```
for (const chunk of retrievedChunks) {  
  const snippet = (chunk.content ?? '').trim();  
  if (!snippet) continue;  
  evidenceChunksById.set(chunk.id, snippet);  
}  
  
const evidenceForFinal = await getEvidenceChunksForFinalAnswer(supabase, slots.map((s) => s.id), evidenceChunksById);  
  
const result = await callFinalAnswer(openaiKey, userMsg, currentSlotStateJson, evidenceForFinal);  
  
Evidence (chunks with ids – cite these with [[quote:uuid]] in your answer):  
---  
${quoteBlock}  
---
```

Фиг. 3.6.3.1

Всички извлечени доказателствени сегменти се акумулират в структурата evidenceChunksById, която обединява резултатите от всички итерации на retrieve фазата (Фиг. 3.6.3.1).

При генериране на крайния отговор тези агрегирани chunks се подават отново към модела. Той може да реферира конкретни доказателства чрез UUID идентификатори, които се маркират в текста във формат [n], след което системата извлича съответните точни цитати от подадените сегменти.

С цел контрол върху контекстния обем е въведен лимит от максимум 80 chunks, които могат да бъдат подадени към финалния етап. Разпределението им се извършва чрез механизма capWithFairAllocation (описан в края на раздел 3.3), който гарантира балансирано покритие между различните подзаявки и итерации, вместо доминиране от един-единствен retrieval резултат.

```
const FINAL_ANSWER_SYSTEM = `You write the final answer to the user's question using only the provided evidence (chunks).
Each chunk has an id in brackets; use [[quote:uuid]] in your answer for every chunk you cite (uuid = chunk id).

Output JSON only:
{
  "final_answer": "your answer text with [[quote:uuid]] placeholders for each citation",
  "cited_snippets": { "uuid-1": "exact verbatim passage from that chunk", "uuid-2": "... " }
}

Rules:
- Base the answer only on the evidence below. Cite every claim with [[quote:uuid]] using the chunk id from the evidence block.
- In cited_snippets, map each cited chunk uuid to the exact verbatim passage you are quoting (one sentence or short passage). Copy from the evidence exactly.
- If some parts of the question could not be answered from the evidence:
  (1) briefly say why (e.g. no evidence in the provided sources); (2) present what you did find with citations; (3) at the end list what could not be found.`;
```

```
await supabase.from('quotes').insert({
  message_id: assistantRow.id,
  page_id: chunk.page_id,
  chunk_id: chunk.id,
  snippet: rawSnippet,
  page_title: page.title ?? '',
  page_path: page.path ?? '',
  domain,
  page_url: fullPageUrl,
  retrieved_in_reasoning_step_id: null,
  owner_id: ownerId,
  citation_order: i + 1,
});
```

Фиг. 3.6.3.2

На финалния етап моделът обработва извлечените evidence chunks и селектира само най-релевантните изречения или фрагменти, които действително подкрепят твърденията в отговора (Фиг. 3.6.3.2). Тези селектирани откъси се записват като записи в таблицата quotes и се реферират в текста чрез маркери от тип [n]. В потребителския интерфейс тези маркери са интерактивни и позволяват инспекция на конкретния източник и контекст, като по този начин се осигурява проследимост и доказуемост на всяко твърдение.

3.6.4. Процес на разсъждение

```
let thoughtProcess: {
  slots: { name: string; type: string; description?: string; dependsOn?: string }[];
  planReason?: string;
  steps: ThoughtStep[];
  iterationCount?: number;
  hardStopReason?: string;
  completeness?: number;
  expandCorpusReason?: string;
  clarifyQuestions?: string[];
  extractionGaps?: string[];
  partialAnswerNote?: string;
} = {
  slots: (planResult?.slots ?? []).map((s) => {
    const o: { name: string; type: string; description?: string; dependsOn?: string } = { name: s.name, type: s.type };
    if (s.description) o.description = s.description;
    if (s.dependsOn) o.dependsOn = s.dependsOn;
    return o;
  }),
  planReason: appendId ? 'Same question, with the new page in the corpus.' : (planResult?.why ?? undefined),
  steps: [],
};

if (thoughtProcess.slots.length > 0) {
  await emit({ thoughtProcess: { ...thoughtProcess } });
}
```

Фиг. 3.6.4.1

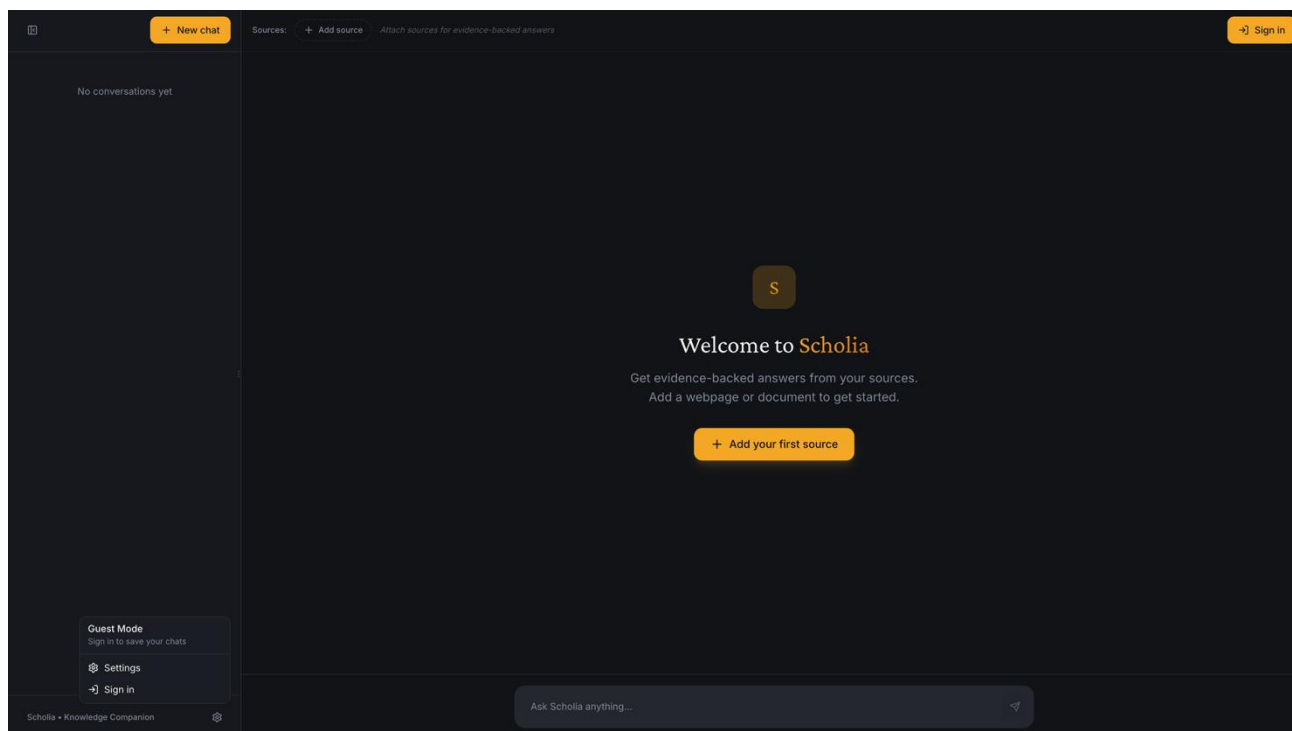
Целият reasoning процес се стриймва към фронтенда в реално време (Фиг. 3.6.4.1). На всяка стъпка се изпраща текущото състояние на изпълнението – кои подзаявки се изпълняват, каква е причината (why) за избраното действие, какви claims са извлечени, както и кои слотове са попълнени или остават непълни след итерацията. Така интерфейсет може да визуализира прогреса и логиката на модела прозрачно, вместо да показва само финалния отговор.

Четвърта глава: Ръководство на потребителя

4.1. Инсталация

Отидете на <https://scholia-hc4t8ibcl-dmyashkovs-projects.vercel.app/>, за да разгледате и тествате уеб сайта. Изтеглетe кода от предадената флашка или от GitHub на <https://github.com/DMyashkov/Scholia>.

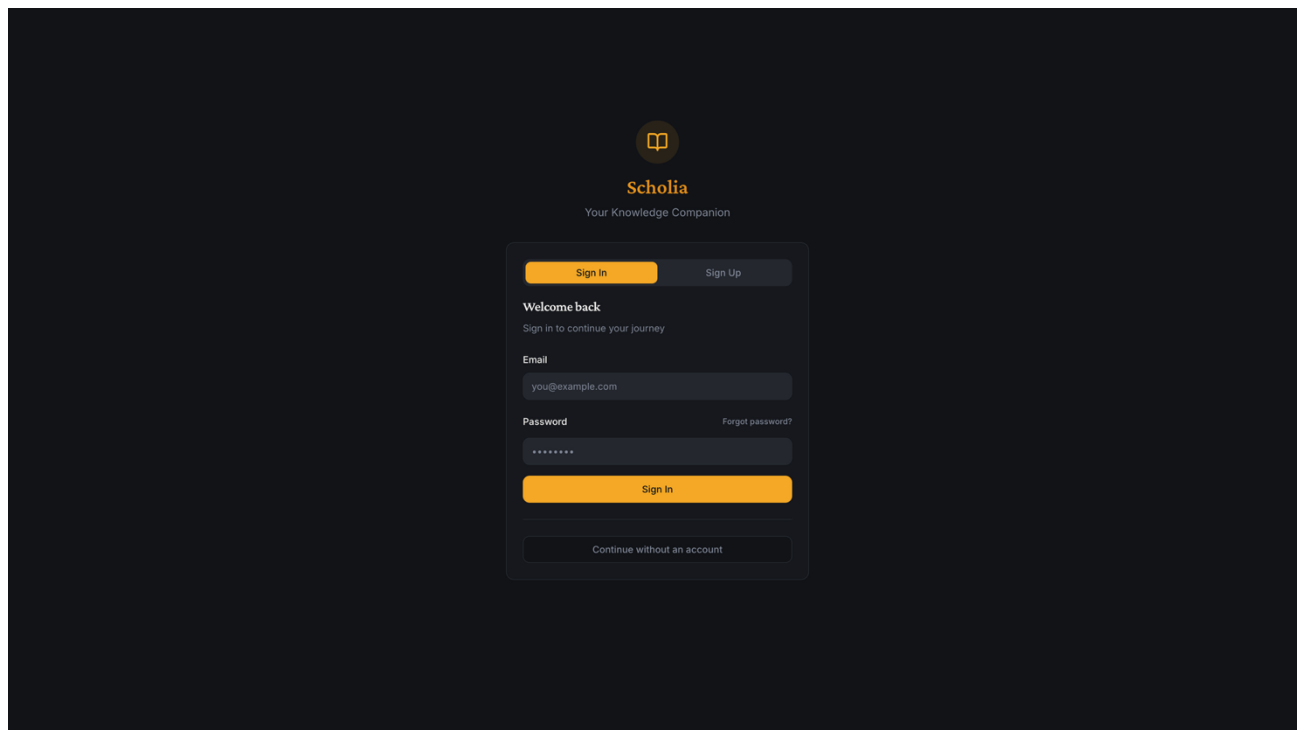
4.2. Автентикация и потребителски профили



Фиг. 4.2.1

При стартиране на платформата първата необходима стъпка е автентикация на потребителя (Фиг. 4.2.1). Вход може да бъде осъществен чрез бутона „Sign in“ в горния десен ъгъл или чрез бутона „Settings“ в долния ляв ъгъл, откъдето се отваря модален прозорец с опциите „Settings“ и „Sign in“.

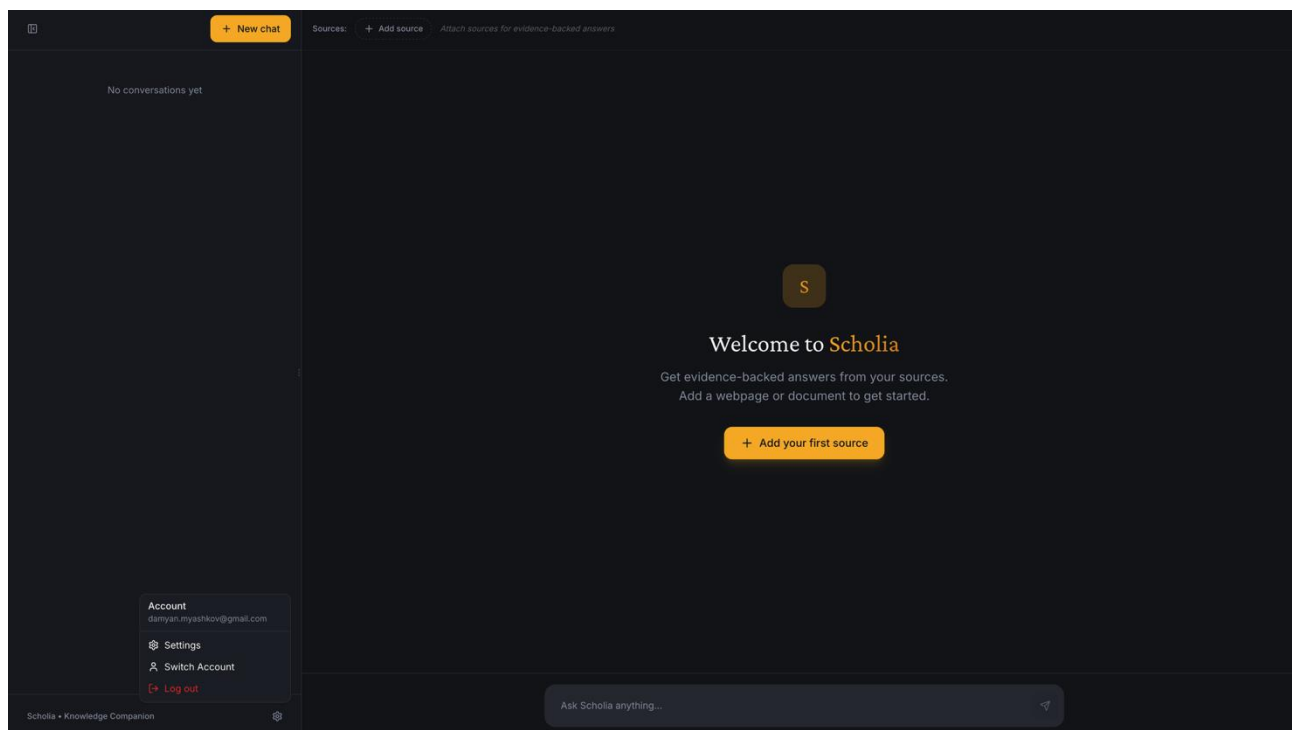
Тъй като приложението все още не е официално публикувано, режимът „guest“ може да бъде наличен или да бъде премахнат в бъдещи версии. В текущата реализация при опит за задаване на въпрос или добавяне на източник без автентикация се визуализира модален прозорец, който изисква вход. Този подход позволява на новите потребители първо да се запознаят с интерфейса и дизайна на платформата, преди да бъдат задължени да преминат през процеса на вход.



Фиг. 4.2.2

След избор на „Sign in“ или „Sign up“ се въвеждат имейл адрес и парола (Фиг. 4.2.2). Може и да се избере опцията „Continue without an account“, която връща потребителя обратно в guest режим.

При създаване на нов акаунт се визуализира съобщение, че трябва да потвърдите имейла си. Чрез изпратения от Supabase Auth линк потребителят може да валидира своя имейл адрес и след това директно да влезе в профила си.



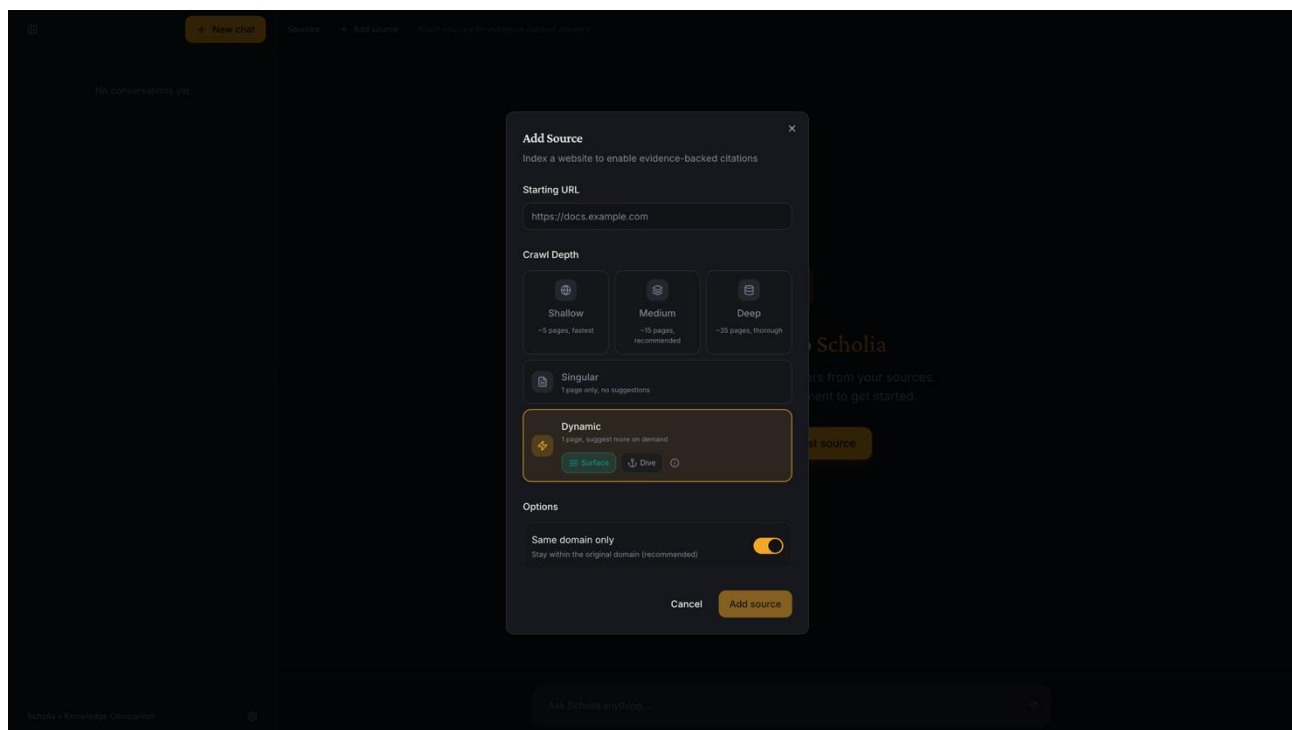
Фиг. 4.2.3

Ако желаете да излезете от своя акаунт, отворете менюто чрез иконката „Settings“ в долния ляв ъгъл. Оттам можете да изберете „Switch Account“ за смяна на профила или „Log out“ за пълно излизане от текущата сесия (Фиг. 4.2.3).

4.3. Създаване на източник

За да добавите източник, изберете „Add your first source“ в централната част на екрана или бутона „Add Source“ в горната лента (top bar).

Ако се опитате да изпратите съобщение, без предварително да сте добавили източник, системата ще блокира действието и ще визуализира модален прозорец с указание да добавите source преди да продължите. За тестване най-лесно и бързо е да ползвате Wikipedia pageове от Wikipedia: Very_short_featured_articles [https://en.wikipedia.org/wiki/Wikipedia:Very_short_featured_articles]. Например в много от примерите в тези инструкции ще видите Wiki article на коня Miss Meyers от този пейдж заради късото му съдържание.



Фиг. 4.3.1

В модалния прозорец можете да изберете между шест режима на обхождане. Статичните режими са Shallow (5 страници), Medium (15 страници), Deep (35 страници) и Singular (1 страница), докато динамичните режими са Dynamic Surface и Dynamic Dive (Фиг. 4.3.1). Всеки от тях определя различен баланс между скорост, обхват и адаптивност на индексирането. Shallow е най-подходящия статичен режим за тестване.

Изберете статичен режим, когато желаете предварително ограничен и предвидим обхват. Той е подходящ не само за малки сайтове, но и за големи платформи с много вътрешни връзки, когато е важно процесът да остане бърз и ресурсно контролиран. При сайтове с хиляди линкове динамичното разширяване може да доведе до по-бавно изпълнение, поради което статичният режим често е по-практичен избор.

Изберете динамичен режим, когато не е ясно предварително кои страници ще бъдат релевантни за конкретния въпрос и желаете корпусът да може да се разширява според нуждите на анализа.

Dynamic Surface е по-бързият вариант и е подходящ за тестване или за първоначално ориентиране. Той оценява потенциалните страници на база



контекста, в който се появява линкът в текущата страница, без да изтегля пълното съдържание на целевата страница предварително.

Dynamic Dive е по-бавен, но по-прецизен режим. Той анализира начален откъс от самата целева страница, което позволява по-информирана оценка на нейната релевантност. Този режим е препоръчителен, когато потребителят няма проблем да изчака по-дълго в замяна на по-вероятно по-точни предложения за разширяване на корпуса.

Изберете "Same domain only" ако искате crawler-a да не излиза от домейна на подадения source.

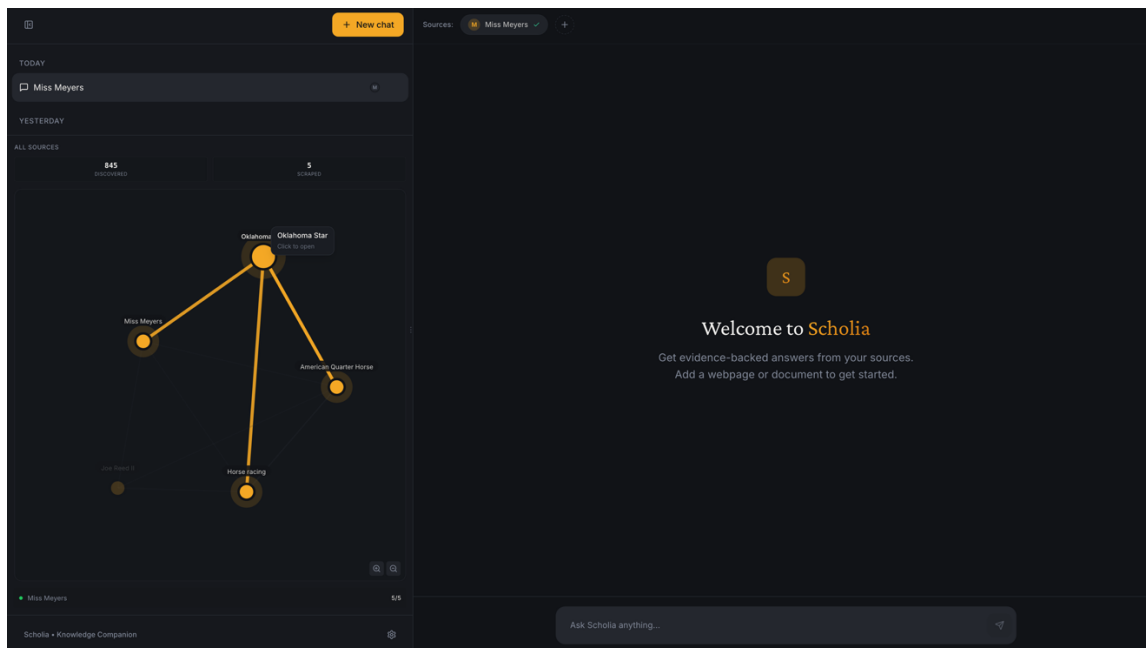
Пуснете обхождането като изберете „Add Source” бутона.

4.4. Работа с граф и менажиране на източници

След добавяне на source процесът по обхождане стартира автоматично. Неговият напредък може да бъде наблюдаван в реално време чрез loading индикатора над граф визуализацията.

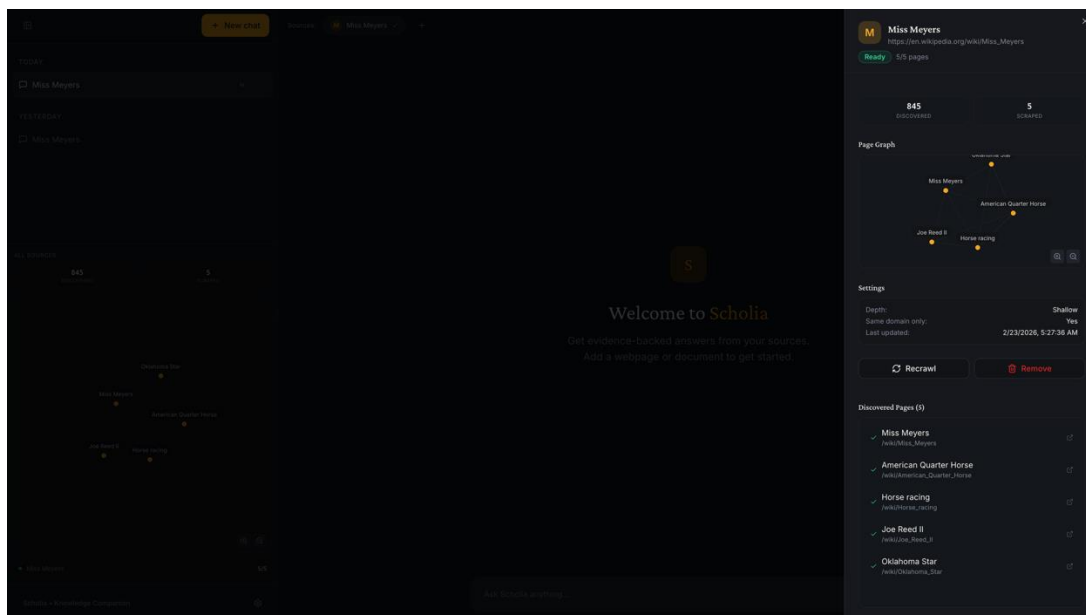
При статичен режим процесът протича в две последователни фази: „Scraping“, при която се извлича съдържанието на страниците, и „Indexing“, при която текстът се разделя и индексира за семантично търсене. И двете фази трябва да приключат успешно, преди да бъде възможно изпращането на съобщение и задаването на въпрос към системата.

Графът визуализира структурата на сайта и връзките между отделните подстраници. За по-добър преглед можете да задържите курсора в дясната част на левия панел и да го разширите чрез плъзгане надясно. Мащабирането се извършва със скрола на мишката или чрез бутоните за увеличение и намаляване в долния ляв ъгъл на графа.



Фиг. 4.4.1

При задържане на курсора върху даден възел се визуализира пълното му име (изведено от заглавието на съответната подстраница), както и се подчертават всички съседни възли, свързани с него (Фиг. 4.4.1). Възлите могат да бъдат премествани чрез плъзгане с цел по-удобно визуално подреждане. При кликване върху възел се отваря съответната уеб страница.



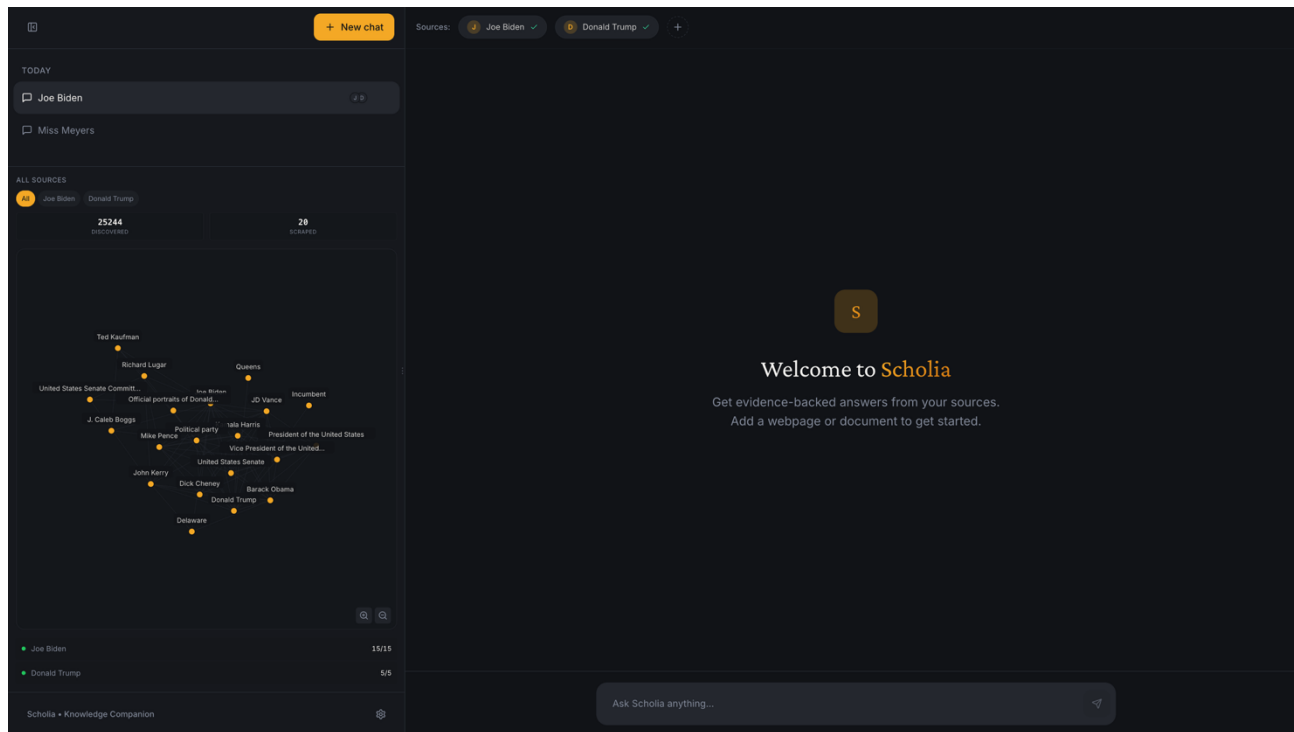
Фиг. 4.4.2



ТЕХНОЛОГИЧНО УЧИЛИЩЕ "ЕЛЕКТРОННИ СИСТЕМИ"
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

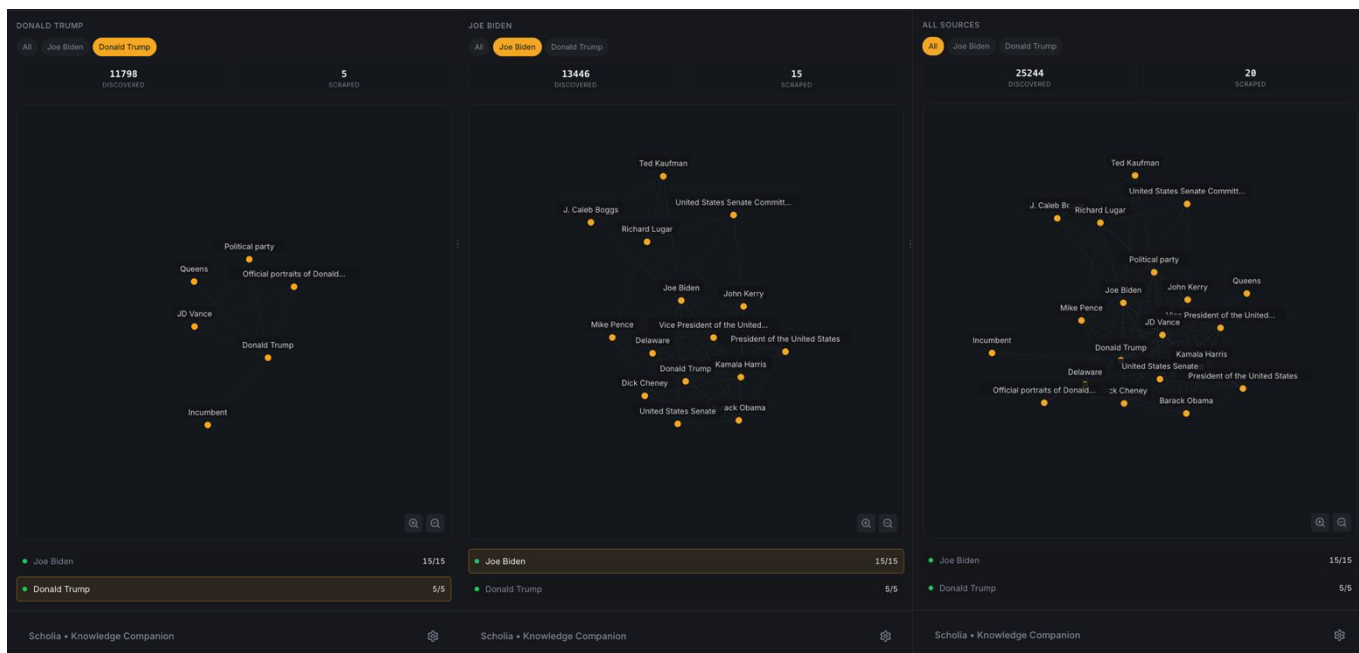
За да видите по-детайлна информация за процеса по обхождане на даден source, кликнете върху него в crawl панела. Ще се отвори изглед със списък на всички индексирани страници, граф визуализация, както и допълнителна информация като дата на обхождане и използваните настройки (Фиг. 4.4.2). От този панел можете също да изтриете source-а от разговора или да стартирате повторно обхождане (recrawl).

В статичен режим граф изгледът показва два основни показателя: „Scraped“, който отразява броя на успешно изтеглените и индексирани страници, и „Discovered“, който показва общия брой срещнати връзки по време на обхождането, независимо дали са били впоследствие обходени.



Фиг. 4.4.3

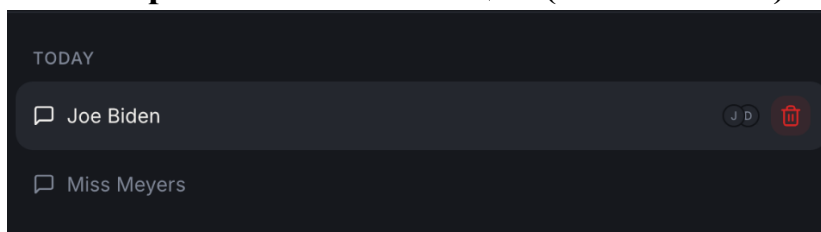
Опитайте да добавите втори source, който е тематично свързан с първия – например страниците в Wikipedia за Donald Trump и Joe Biden. След като процесът по обхождане приключи, ще забележите, че в граф изгледа възлите от двата източника са свързани помежду си, тъй като страниците съдържат взаимни препратки.



Фиг. 4.4.4

В граф визуализацията можете да филтрирате кои възли да бъдат показвани. В горната част ще се появи лента с опции като „All“, „Donald Trump“ и „Joe Biden“, чрез които можете да избирате кой източник да бъде визуализиран. Същото филтриране може да се извърши и чрез селектиране или деселектиране на съответните source-ове в списъка под граф изгледа (Фиг. 4.4.4).

4.5. Менажиране на чат инстанции (conversations)

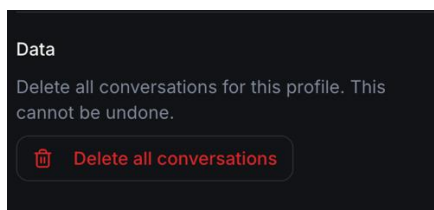


Фиг. 4.5.1



Натиснете бутона „New Chat“, за да изчистите текущия изглед и да започнете нов разговор. При добавяне на source автоматично се създава нова чат инстанция.

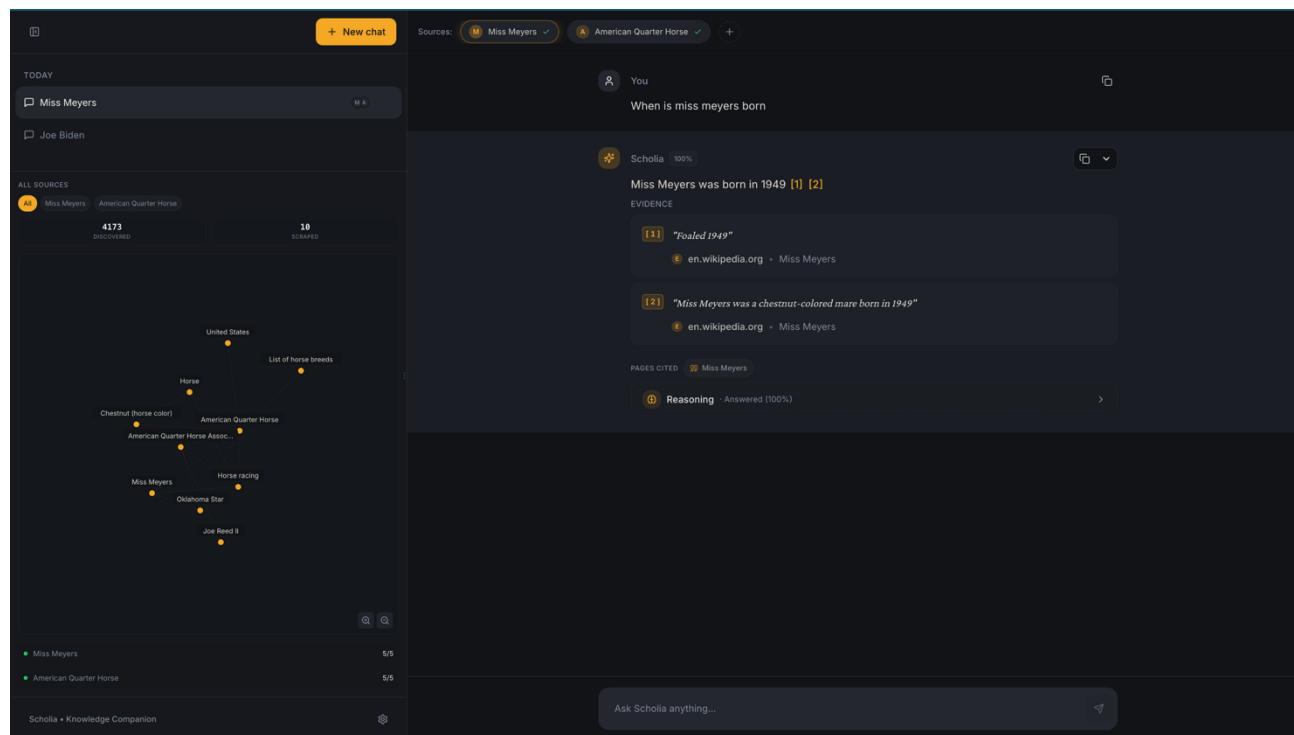
В левия панел, когато даден чат е избран, ще видите малки кръгли индикатори с първата буква на всеки source, свързан с този разговор. Ако задържите курсора върху съответната чат инстанция, отдясно ще се появи бутон за изтриване, чрез който можете да премахнете конкретния conversation (Фиг. 4.5.1).



Фиг. 4.5.2

Ако кликнете в долния ляв ъгъл върху „Scholia Knowledge Companion“ и изберете Settings, ще получите възможност да изтриете всички разговори, свързани с вашия акаунт (Фиг. 4.5.2). При по-голям брой conversation-и процесът може да отнеме няколко секунди, през които е необходимо да изчакате завършването му.

4.6. Общуване с чатбота

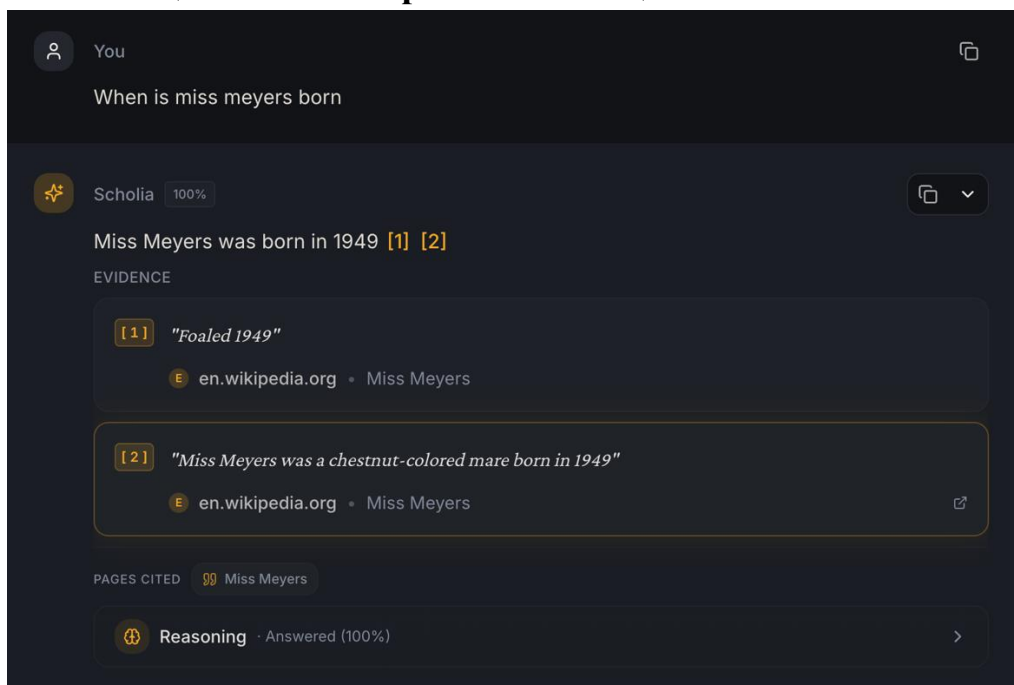


Фиг. 4.6.1

Тъй като основната цел на приложението е генериране на отговори, подкрепени с конкретни цитирани доказателства, не е възможно да зададете въпрос преди да добавите source. След като източникът бъде индексирен, можете да използвате чат полето, за да задавате въпроси, свързани със съдържанието на подадените страници. Системата ще генерира отговор, като прикрепи възможно най-много релевантни цитати от индексирания корпус (Фиг. 4.6.1).

Можете например да добавите статия от Wikipedia и да зададете въпрос, свързан с някоя от страниците, които виждате в граф визуализацията. В интерфейса ще се визуализира и изчислен процент, който показва до каква степен въпросът е бил покрит от наличната информация в корпуса.

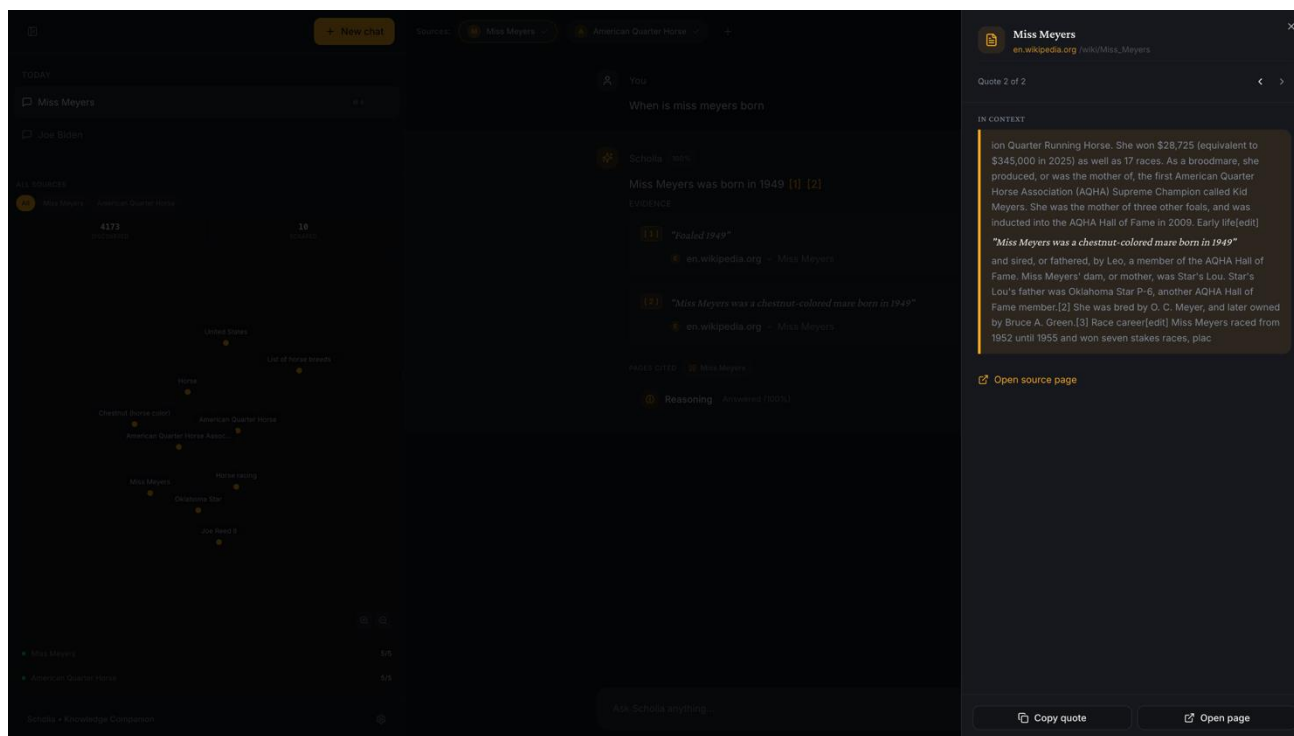
4.7. Работа с цитати и копиране на съобщения



Фиг. 4.7.1

След като чатботът върне отговор, използваните цитати обикновено са маркирани в текста с индекси от вида [n] (съответстващи на вътрешен UUID) и са изписани под съобщението със същия номер и пълния текстов откъс (Фиг. 4.7.1).

При кликване върху даден индекс [n] в отговора или директно върху самия цитат ще се отвори десен панел с допълнителна информация за съответния quote, включително източника и контекста, от който е извлечен (Фиг. 4.7.2).

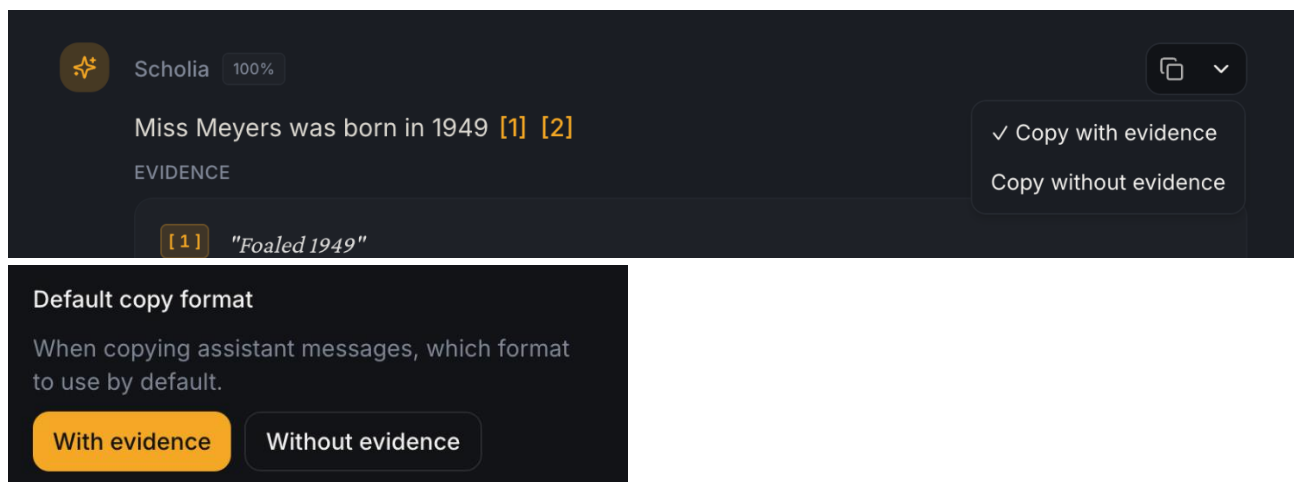


Фиг. 4.7.2

В десния панел обикновено се визуализира цитата в контекст, заедно с няколко изречения преди и след него от съответната уеб страница. Показва се именно извлеченото (scraped) съдържание, поради което в някои случаи е възможно текстът да не съвпада напълно със стандартната визуална подредба на сайта - например ако цитатът се намира в самото начало или край на страницата.

В този панел можете да използвате бутона „Copy quote“, за да копирате цитата, или „Open page“, за да отворите подстраницата, от която произлиза. Ако сайтът поддържа подобна функционалност и цитатът не се намира в нестандартна секция (например страничен панел вместо основното съдържание), страницата ще се отвори директно на съответната позиция и ще маркира цитирания текст.

В панела са налични и стрелки наляво и надясно (част от интерфейса), чрез които можете да преминавате последователно между всички цитати, използвани в текущия разговор.

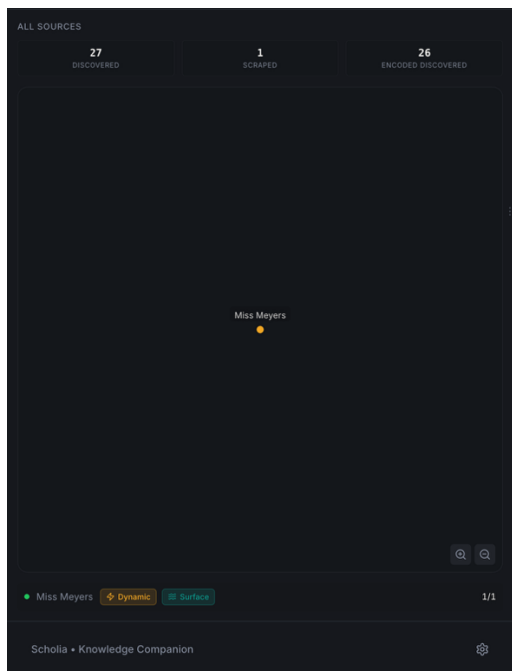


Фиг. 4.7.3

За да копирате съобщение, използвайте бутона за копиране в горния десен ъгъл на отговора. До него ще видите стрелка за избор на режим, чрез която можете да превключвате между опциите „Copy without evidence“ и „Copy with evidence“ (Фиг. 4.7.3).

При избор на „Copy without evidence“ текстът ще бъде копиран без включените цитати под съобщението и без референтните индекси (UUID) в самия текст. Препоръчително е да изпробвате и двата варианта, за да видите разликата. Тази настройка може да бъде променяна и от менюто Settings.

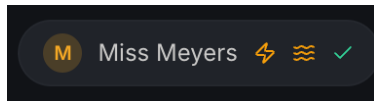
4.8. Динамичен режим



Фиг. 4.8.1

За да работите в динамичен режим, добавете source с избран динамичен режим и стартирайте обхождането (Фиг. 4.8.1). В този случай loading индикаторът ще премине през три фази: „Scraping“, „Indexing“ и „Encoding Discovered“. Последната фаза означава, че откритите, но все още необходими страници се кодират семантично, така че впоследствие да могат да бъдат предлагани като потенциални разширения на корпуса.

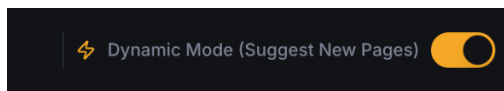
В менюто Settings можете да увеличите броя на „suggestion candidates“ от 5 на 10. Това предоставя по-голям набор от възможни предложения, измежду които системата може да избере, което може леко да подобри точността на препоръките. Увеличаването на тази стойност обаче води до по-висока консумация на AI кредити.



Фиг. 4.8.2

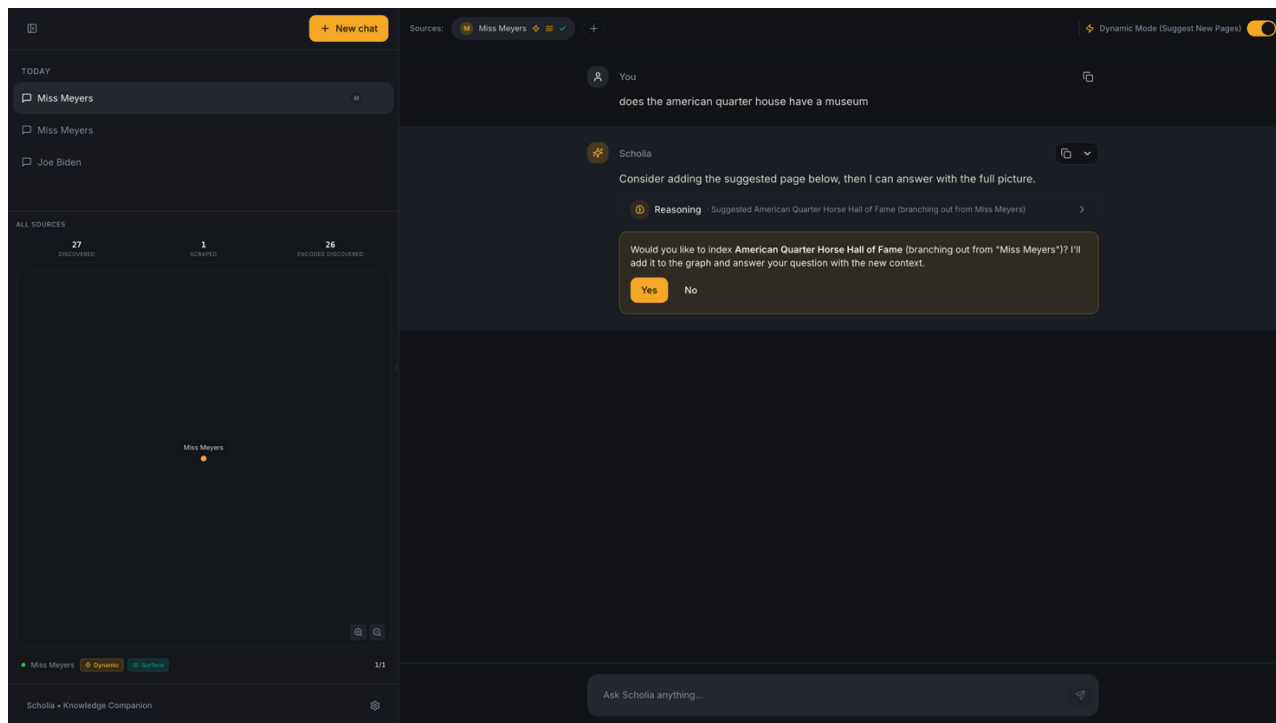
До динамичните source-ове ще виждате индикатори (badge-ове) в списъка под граф изгледа, които показват дали източникът е в динамичен режим и кой тип е избран – Surface или Dive (Фиг. 4.8.2).

В горната лента на sources иконата със светкавица означава, че режимът е динамичен. Иконата с вълни обозначава Surface режим, а иконата с котва – Dive режим.



Фиг. 4.8.3

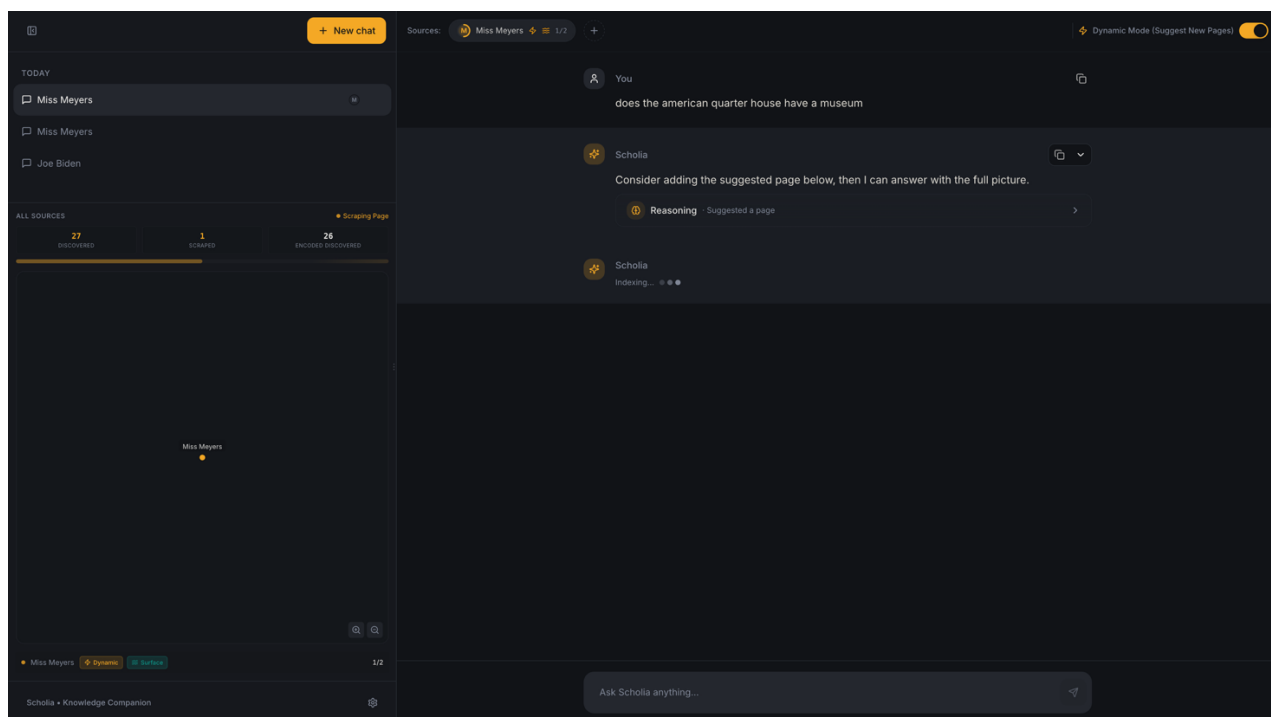
В горната дясна част на интерфейса е налична опция dynamicMode, чрез която можете да изключите динамичния режим (Фиг. 4.8.3). При деактивиране системата ще спре да предлага нови страници за разширяване на корпуса и ще работи единствено с вече индексираното съдържание.



Фиг. 4.8.4

За да тествате динамичното разширяване, задайте въпрос, който изисква информация от свързана страница. Например добавете статията за Miss Meyers в Wikipedia и попитайте: „Does the American Quarter Horse have a museum?“ (Фиг. 4.8.4).

Тъй като тази информация се намира в отделна, но свързана страница (напр. American Quarter Horse Hall of Fame and Museum), системата ще предложи релевантен discovered page и при одобрение или автоматичен режим, ще разшири корпуса, за да отговори по-пълно на въпроса.

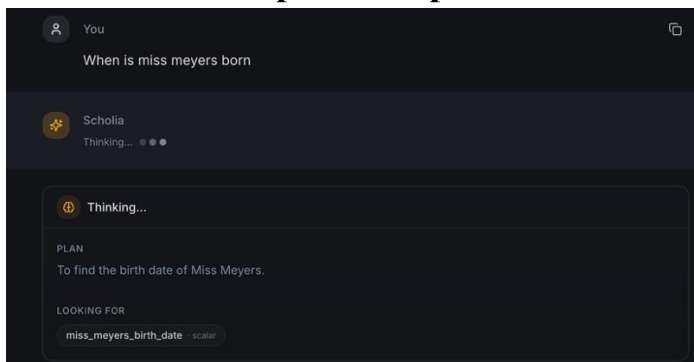


Фиг. 4.8.5

При предложение за добавяне на страница (suggestion) изберете Yes, ако сте съгласни с решението на модела. След приемане страницата ще бъде обходена и индексирана, а откритите ѝ съседни връзки ще бъдат включени в Encoding Discovered, за да могат да се използват за следващи предложения. Когато този процес приключи, loading индикаторът преминава в нова фаза Responding, в която системата финализира отговора (Фиг. 4.8.5).

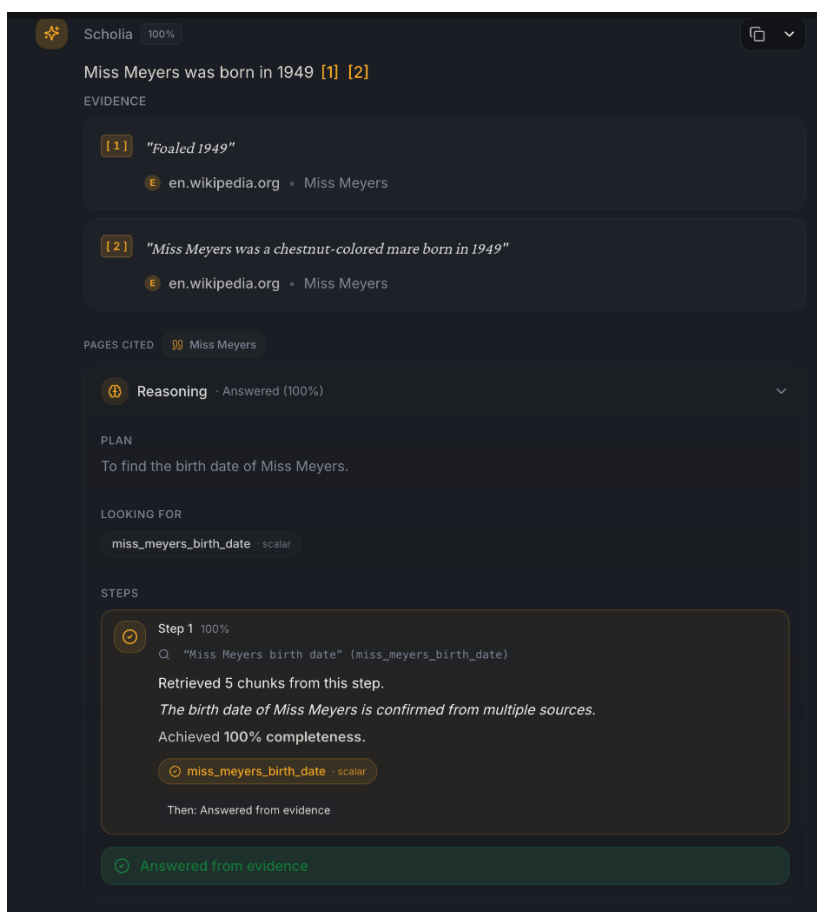
Преди да приемете suggestion, можете да прегледате и междинния (incomplete) reasoning процес, за да видите какво вече е установено и защо се предлага разширяване на корпуса. Когато отговорът бъде готов, текущият резултат ще бъде допълнен с допълнителни reasoning стъпки и финален отговор. В интерфейса ще виждате ясно разделение, маркирано с етикет от типа „Scraped [име на страницата]“, което показва кога е добавена и обработена новата страница в рамките на разширения корпус.

4.9. Инспекция на процес на разсъждаване на модела



Фиг. 4.9.1

По време на зареждането ще се визуализира междинният thought process на модела (Фиг. 4.9.1). След като заявката приключи, можете да разгънете reasoning секцията чрез кликуване, за да видите подробностите по процеса на разсъждение (Фиг. 4.9.2).

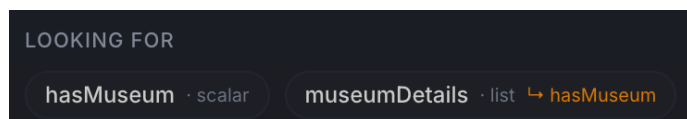


Фиг. 4.9.2



В тази секция ще видите и слотовете – структурирани променливи, които моделът е идентифицирал като необходими за отговора, както и начина, по който те са били попълвани в хода на итерациите на reasoning процеса.

За да видите описанието на конкретен слот, задръжте курсора върху него – ще се появи tooltip с генерираната дефиниция и допълнителна информация за ролята му в анализа.



Фиг. 4.9.3

Зависимостите между променливите (слотовете) са маркирани в жълто и също се визуализират в tooltip при задържане на курсора. Това позволява да се проследи кои слотове зависят от стойностите на други и как се изгражда логическата структура на отговора (Фиг. 4.9.3).

Процентът, който показва до каква степен въпросът е бил покрит, се изчислява въз основа на броя и вида на попълнените слотове. Колкото по-голяма част от идентифицираните слотове са успешно запълнени, толкова по-висок е процентът на покритие на заявката.

Заклучение

Дипломната работа разглежда проблема за систематичното, контролируемо и доказуемо уеб проучване в контекста на съвременните езикови модели. В хода на изследването бе установено, че макар големите езикови модели да улесняват синтезирането и структурирането на информация, те не гарантират измерим обхват на анализа и пълна проследимост на всяко твърдение. От своя страна класическите инструменти за обхождане на уеб съдържание осигуряват систематично събиране на данни, но не предлагат механизъм за семантично търсене и аргументирано генериране на отговори.

В отговор на този проблем бе проектирана и реализирана системата Scholia, която интегрира автоматизирано обхождане, векторно индексирание, Retrieval-Augmented Generation механизъм и многостепенен алгоритъм за структурирано разсъждение. Основният принос на разработката е създаването на изследователска среда с ясно дефиниран корпус от източници, в която всяко твърдение е обвързано с конкретен текстов откъс.

От техническа гледна точка системата изгражда графова структура на страниците чрез обхождане в ширина, индексира съдържанието чрез семантични векторни представяния и реализира итеративен reasoning процес със слотове и зависимости, който позволява измерима оценка на степента на покритие на въпроса. Динамичният режим допълнително осигурява адаптивно разширяване на корпуса чрез семантично подбрани предложения, което подобрява гъвкавостта на анализа.

В бъдеще развитието на системата може да продължи в няколко практически направления. Едно от тях е интегриране на механизъм за web search, който да позволява автоматично откриване на нови релевантни източници извън първоначално зададения домейн. Това би разширило приложимостта на системата при изследователски задачи, при които не е известен конкретния начален източник.

Друго направление е добавяне на поддръжка за допълнителни типове файлове, като PDF, DOCX и други структурирани документи. Това би позволило системата да се използва не само за уеб страници, но и за анализ на академични



публикации, отчети и вътрешни документи, което значително би разширило обхвата ѝ.

В заключение, разработената система демонстрира, че чрез внимателно съчетаване на уеб технологии и съвременни езикови модели може да бъде изградена среда за интелигентно и проследимо уеб проучване. Scholia поставя основа за по-прозрачни и доказуеми AI инструменти, които съчетават автоматизация с контрол върху източниците и аргументацията.



Използвана литература:

[2.5.1 - 1] “Single database vs database per user: there are two main ways to achieve data isolation for each tenant. You can have **a single, shared database where every table has an organization_id** and every read and write will include organization_id. Or you can create a separate database for each user with fully separate database instances or different schemas inside a single database.” - https://www.flightcontrol.dev/blog/ultimate-guide-to-multi-tenant-saas-data-modeling?utm_source=chatgpt.com

[3.3 – 1] HNSW indexing is so useful for doing a similarity search, or approximate nearest neighbour (ANN) search, in vector embedding space. - <https://medium.com/@wtaisen/hnsw-indexing-in-vector-databases-simple-explanation-and-code-3ef59d9c1920>



Съдържание:

Увод.....	1
Първа глава: Съществуващи решения.....	3
1.1. Google / Bing: Стандартното проучване в уеб среда (процес и проблеми).....	3
1.2. Подобни системи и продукти.....	4
1.2.1. ChatGPT, Claude & Gemini – Стандартни големи езикови модели като инструмент за проучване: възможности и ограничения.....	4
1.2.1.1. Пример, в който излизат наяве недостатъците.....	4
1.2.1.2. Структурни ограничения.....	6
1.2.2. Специализирани LLM системи с предварително подбран или качен набор от документи.....	8
1.2.2.1. Домейн-специфични GPT системи.....	9
1.2.2.2. Системи за чат върху качени документи.....	9
1.2.2.3. Обобщение.....	10
1.2.3. Perplexity и сходни търсецо-ориентирани AI системи с цитиране на източници.....	10
1.2.4. Класически web scraping и crawler инструменти: възможности за интеграция с езикови модели.....	12
1.3. Извод от прегледа: какви пропуски остават и как Scholia ги попълва.....	14
Втора глава: Изисквания и проектиране.....	15
2.1. Функционални изисквания.....	15
2.1.1. Потребителска автентикация и изолация на данни.....	15
2.1.2. Създаване и конфигуриране на източник (Source).....	15
2.1.3. Автоматизирано обхождане на уеб страници.....	15
2.1.4. Индексиране и семантично представяне на съдържание.....	15
2.1.5. Задаване на въпроси върху избран корпус.....	15
2.1.6. Генериране на отговор с цитати.....	15
2.1.7. Динамично разширяване на корпуса.....	15
2.1.8. Визуализация на граф и прогрес.....	16
2.2. Нефункционални изисквания.....	16
2.2.1. Проследимост и проверимост.....	16
2.2.2. Контролиран обхват на анализа.....	16



2.2.3. Спазване на уеб стандарти и етика.....	16
2.2.4. Ефективност при семантично търсене.....	16
2.2.5. Реално време и синхронизация.....	16
2.3. Алгоритъм и архитектура: резюмирани.....	16
2.4. Технологии - език, framework, БД, векторно хранилище.....	19
2.4.1. React + TypeScript + Vite.....	19
2.4.2. Tailwind CSS + ShadCN / Radix UI.....	20
2.4.3. Supabase (PostgreSQL + Auth + Realtime + Edge Functions).....	21
2.4.4. TanStack Query.....	22
2.4.5. Pgvector.....	22
2.4.6. OpenAI - text-embedding-3-small, gpt-4o-mini.....	22
2.4.7. Cheerio + node-fetch + robots-parser.....	23
2.4.8. D3.js.....	24
2.4.9. Vercel.....	24
2.5. Структура на базата данни.....	24
2.5.1. Таблица за потребителски настройки.....	25
2.5.2. Таблици, свързани с цитираните източници - sources, pages & crawl jobs.....	26
2.5.3. Таблица, свързана с RAG – Chunks.....	27
2.5.4. Таблица, свързана с графа - Page Edges.....	27
2.5.5. Таблици свързани с чат – conversations, messages и quotes.....	27
2.5.6. Таблици, свързани с процеса на разсъждение на модела – reasoning_steps, slots, slot_items, claim_evidence и reasoning_subqueries.....	28
Трета глава: Реализация на системата Scholia.....	32
3.1. Обхождане на сайтове.....	32
3.1.1. Robots Parser.....	33
3.1.2. Алгоритъм за обхождане на източници чрез BFS (скелет).....	34
3.1.3. Извличане на линковете и нормализирането им.....	35
3.1.4. Crawling.....	37
3.1.5. Recrawling.....	38
3.2. RAG Indexing.....	38
3.2.1. Chunking.....	39
3.2.2. Batch embedding.....	40



3.3. Извличане на релевантни текстови сегменти чрез RAG и разпределение на chunk-овете за всяка заявка.....	40
3.4. Динамичен режим.....	44
3.4.1. Encoded Discovered – Dynamic Surface и Dive Имплементация.....	46
3.4.1.1. Dynamic Surface.....	46
3.4.1.2. Dynamic Dive.....	47
3.4.2. Suggestions.....	47
3.4.3. Добавяне на страница към динамичен source.....	49
3.4.4. Recrawl на динамичен source.....	49
3.5. Supabase Realtime.....	49
3.5.1. Crawl_jobs канал.....	50
3.5.2. Page_edges канал – Граф.....	50
3.6. Главен Алгоритъм – Алгоритъм за многостепенно разсъждение.....	53
3.6.1. Фаза на планиране.....	53
3.6.2. Цикъл на мислене и итерациите му.....	56
3.6.2.1. BROAD срещу TARGETED subquery-та.....	58
3.6.2.2. Кога всеки слот спира да се query-ва.....	59
3.6.3. Финален отговор.....	61
3.6.4. Процес на разсъждение.....	63
Четвърта глава: Ръководство на потребителя.....	64
4.1. Инсталация.....	64
4.2. Автентикация и потребителски профили.....	64
4.3. Създаване на източник.....	66
4.4. Работа с граф и менажиране на източници.....	68
4.5. Менажиране на чат инстанции (conversations).....	71
4.6. Общуване с чатбота.....	72
4.7. Работа с цитати и копиране на съобщения.....	73
4.8. Динамичен режим.....	75
4.9. Инспекция на процес на разсъждаване на модела.....	79
Заклучение.....	81
Използвана литература.....	83